



US 20250286641A1

(19) **United States**

(12) **Patent Application Publication**
Ahn et al.

(10) **Pub. No.: US 2025/0286641 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **SYNCHRONIZING SYSTEMS-ON-CHIP
USING GPIO TIMESTAMPS**

G06F 13/24 (2006.01)

G06F 13/40 (2006.01)

(71) Applicant: **Snap Inc.**, Santa Monica, CA (US)

(52) **U.S. Cl.**

CPC **H04J 3/065** (2013.01); **G02B 27/017**

(2013.01); **G06F 1/10** (2013.01); **G06F 1/12**

(2013.01); **G06F 13/24** (2013.01); **G06F**

13/4068 (2013.01); **G02B 2027/0178**

(2013.01); **G06F 2213/0026** (2013.01); **G06F**

2213/0038 (2013.01)

(21) Appl. No.: **19/217,558**

(57)

ABSTRACT

(22) Filed: **May 23, 2025**

Related U.S. Application Data

(63) Continuation of application No. 18/644,932, filed on Apr. 24, 2024, which is a continuation of application No. 17/496,261, filed on Oct. 7, 2021, now Pat. No. 12,021,611.

An electronic eyewear device includes first and second systems-on-chip (SoCs) having independent time bases. The first and second SoCs are connected by a shared general purpose input/output (GPIO) connection and an inter-SOC interface. The first and second SoCs are synchronized to each other by the first SoC asserting the shared GPIO connection to the second SoC where assertion of the message to the shared GPIO connection triggers an interrupt request (IRQ) at the second SoC. The first SoC records a first timestamp for assertion of the message to the GPIO connection, and the second SoC records a second timestamp of receipt of the IRQ. The first SoC sends a message including the first timestamp to the second SoC over the inter-SoC interface. The second SoC calculates a clock offset between the first and second SoCs as a difference between the first and second timestamps.

Publication Classification

(51) **Int. Cl.**

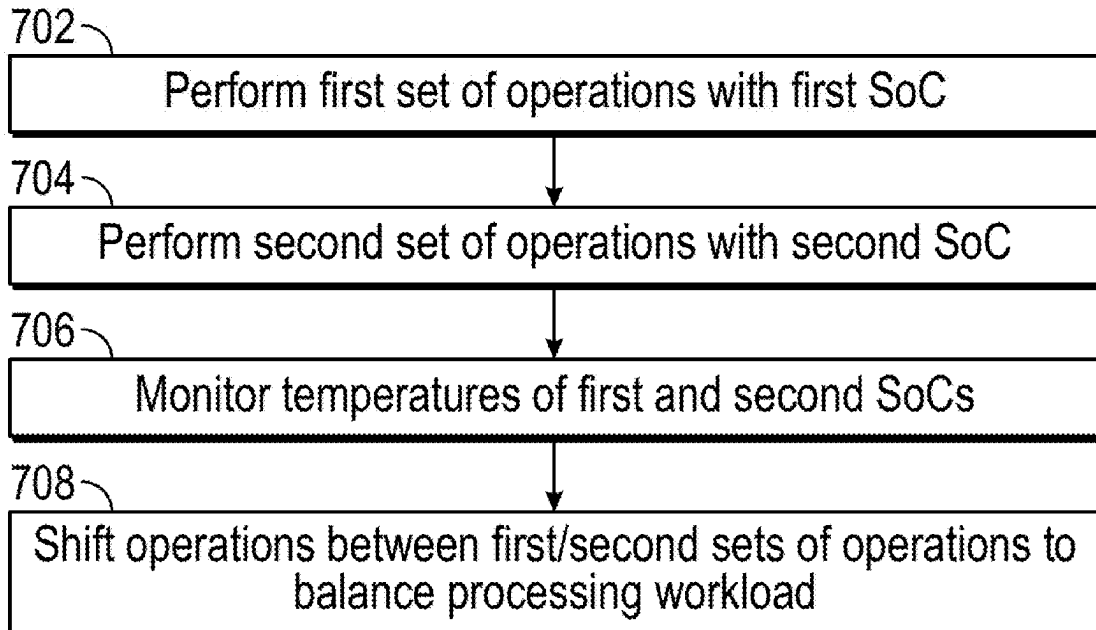
H04J 3/06 (2006.01)

G02B 27/01 (2006.01)

G06F 1/10 (2006.01)

G06F 1/12 (2006.01)

700 ↗



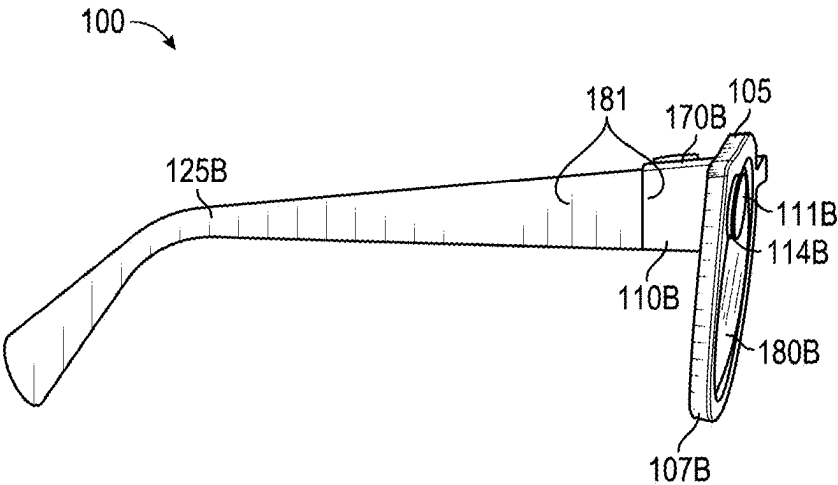


FIG. 1A

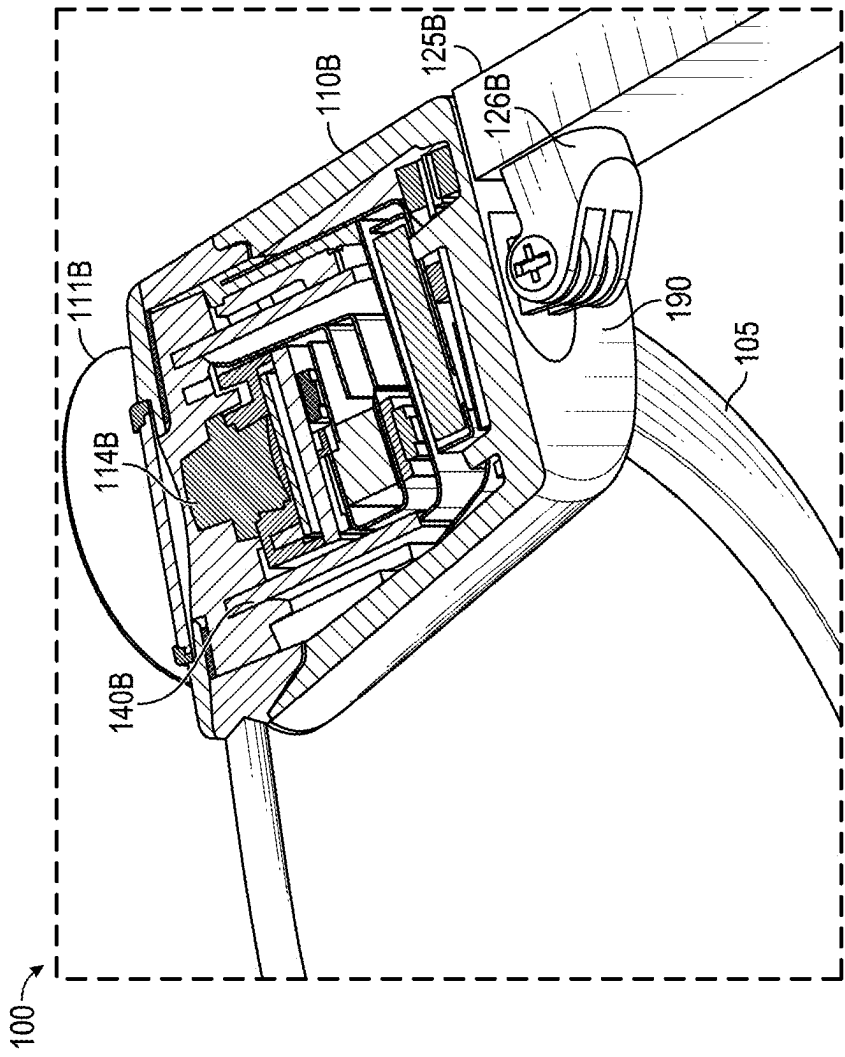


FIG. 1B

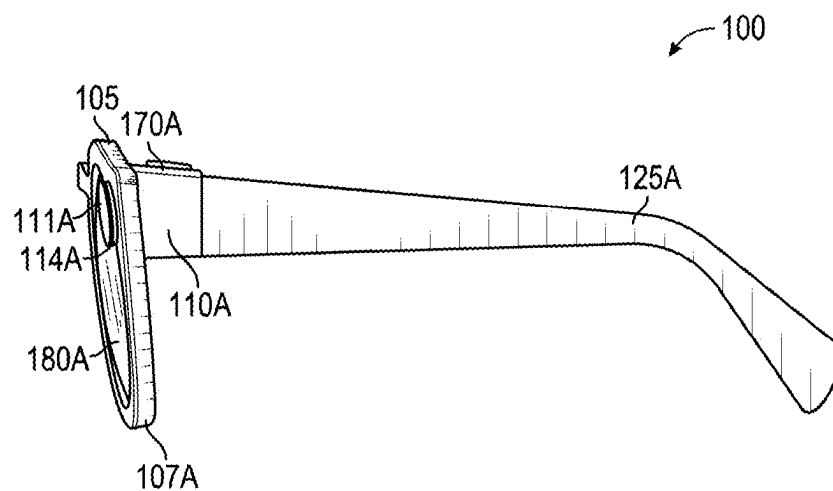


FIG. 1C

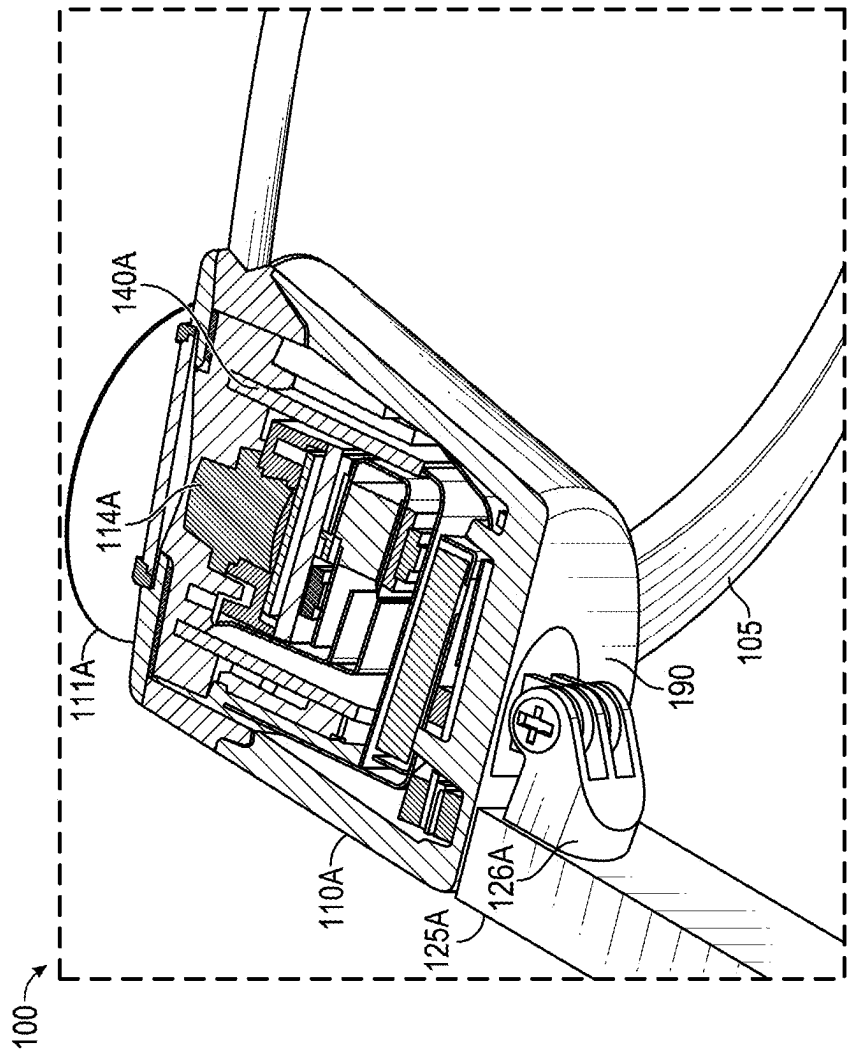


FIG. 1D

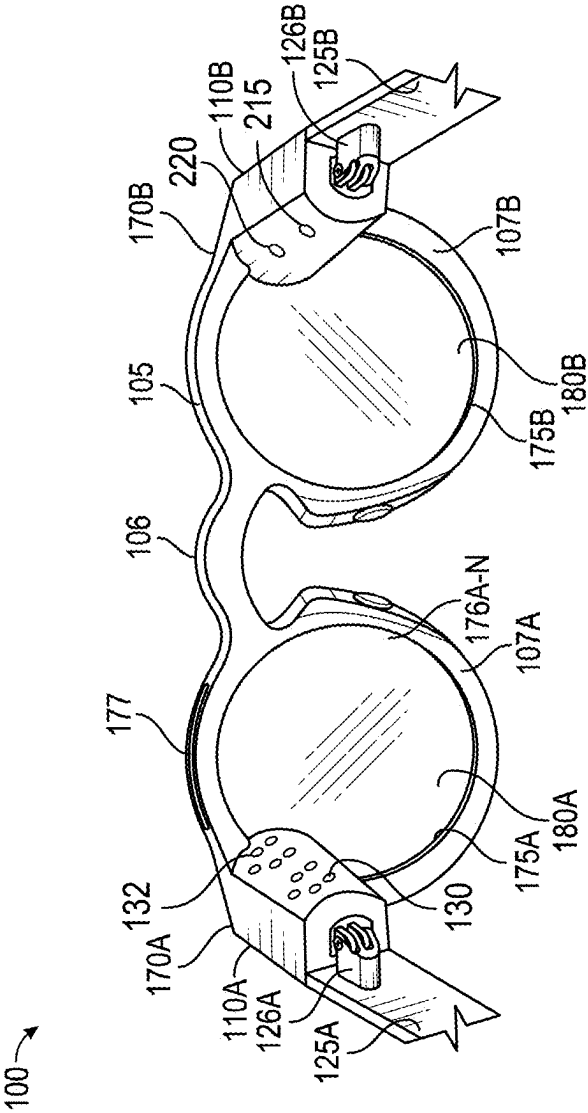


FIG. 2A

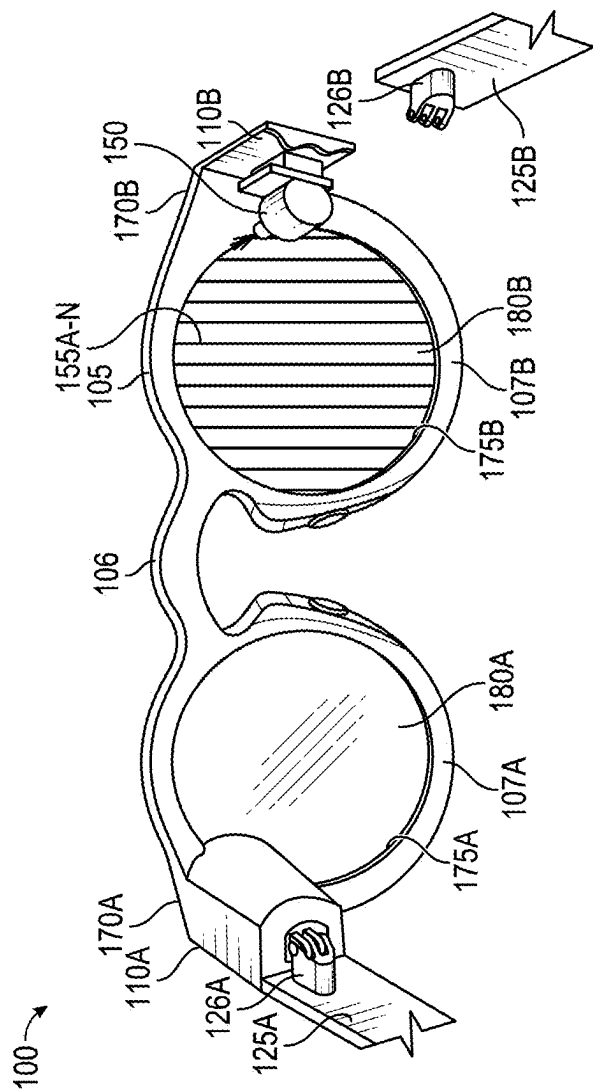


FIG. 2B

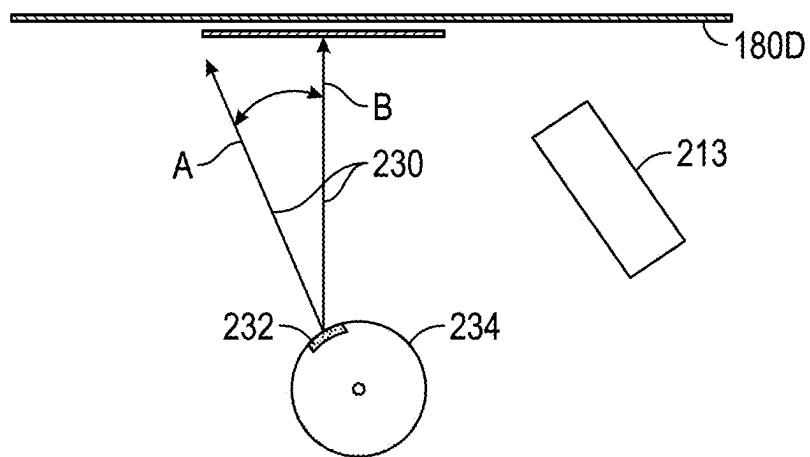


FIG. 2C

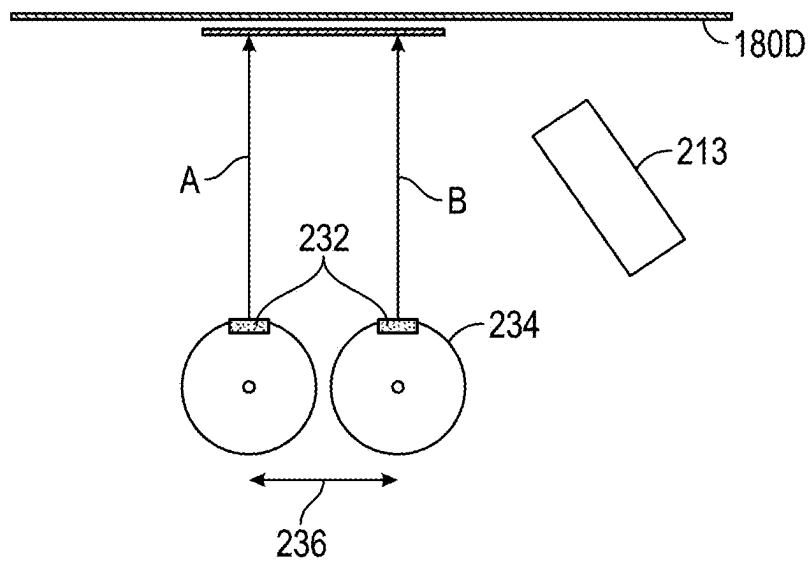


FIG. 2D

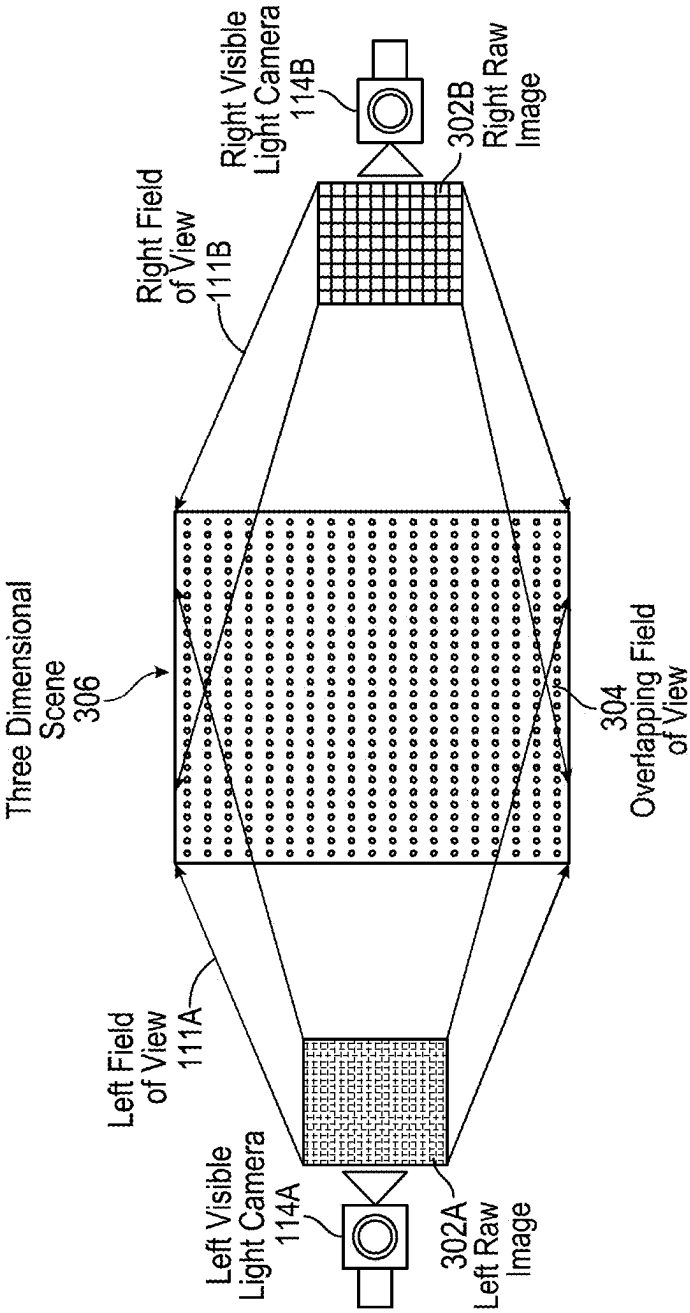


FIG. 3

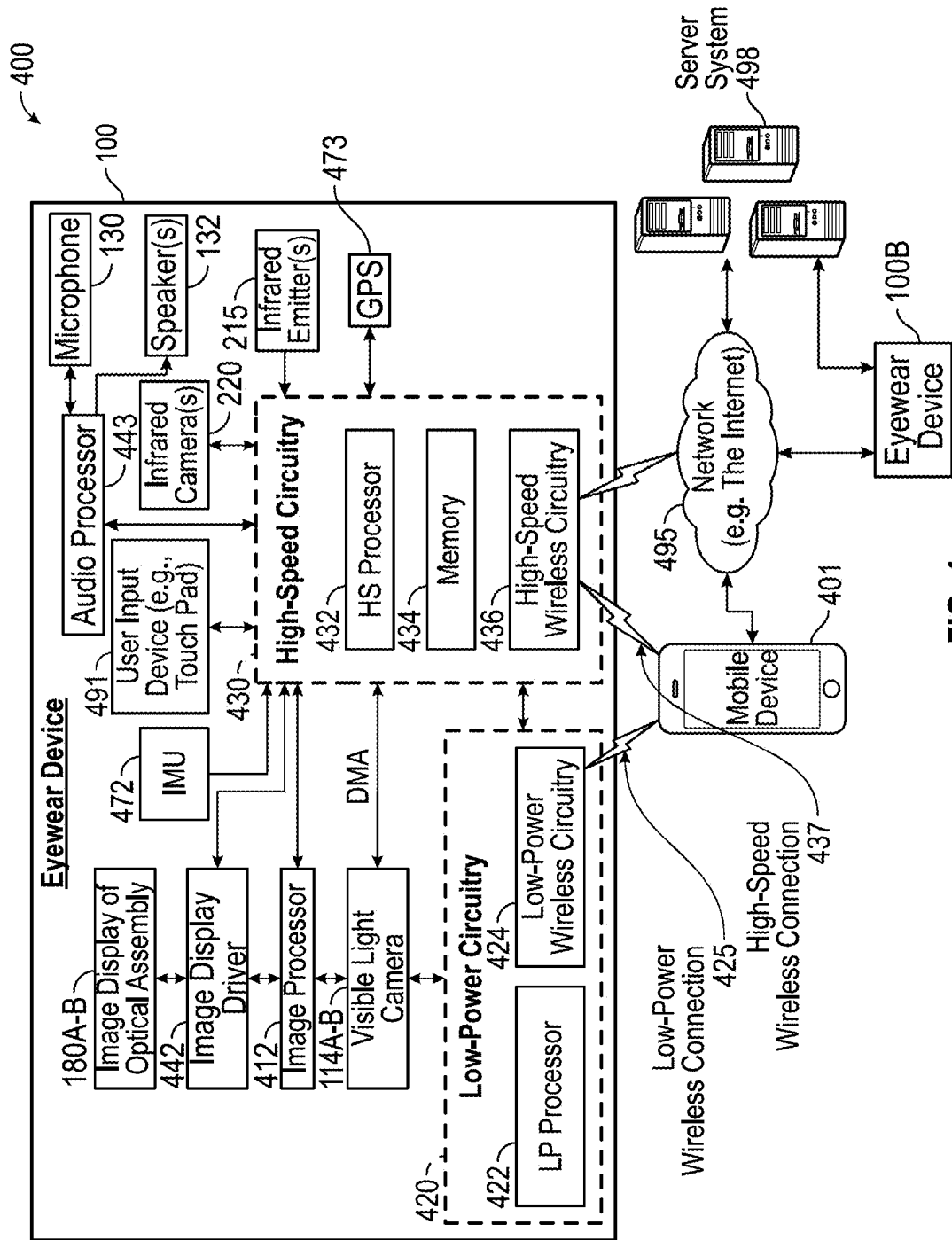


FIG. 4

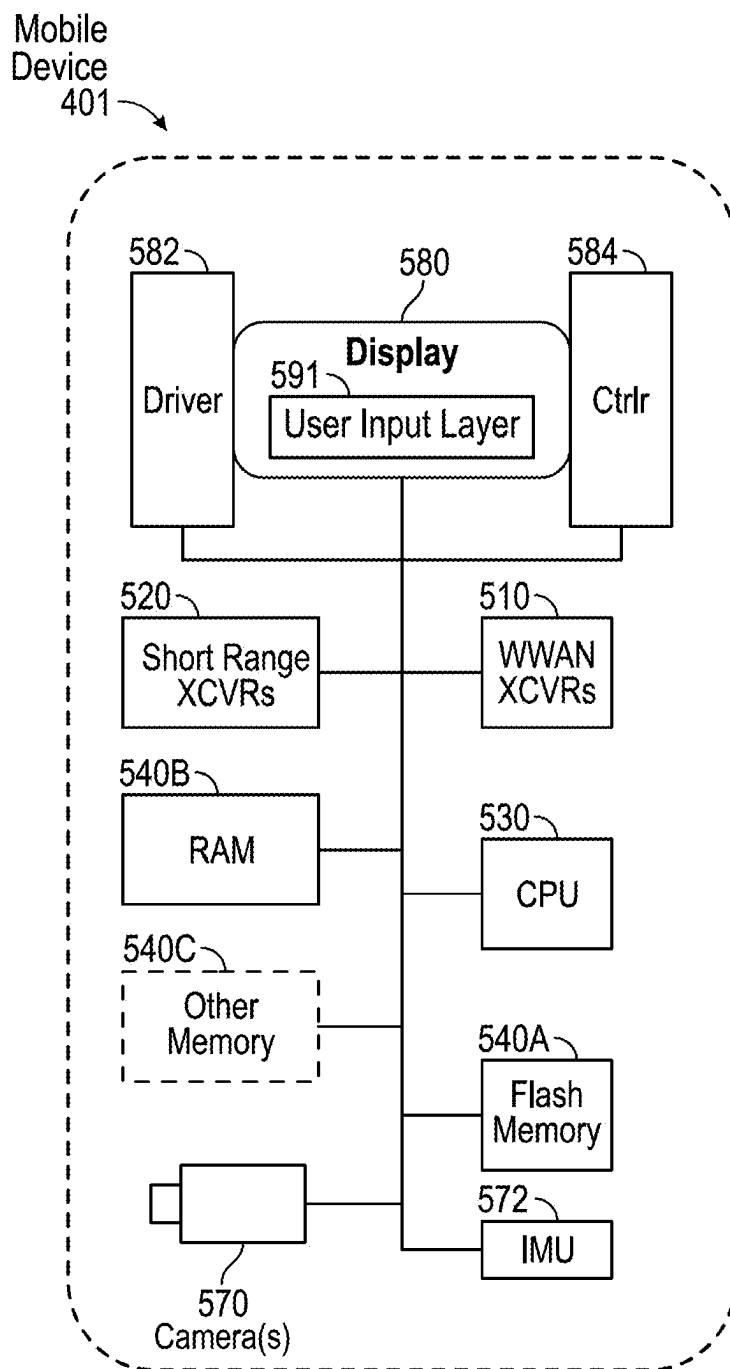


FIG. 5

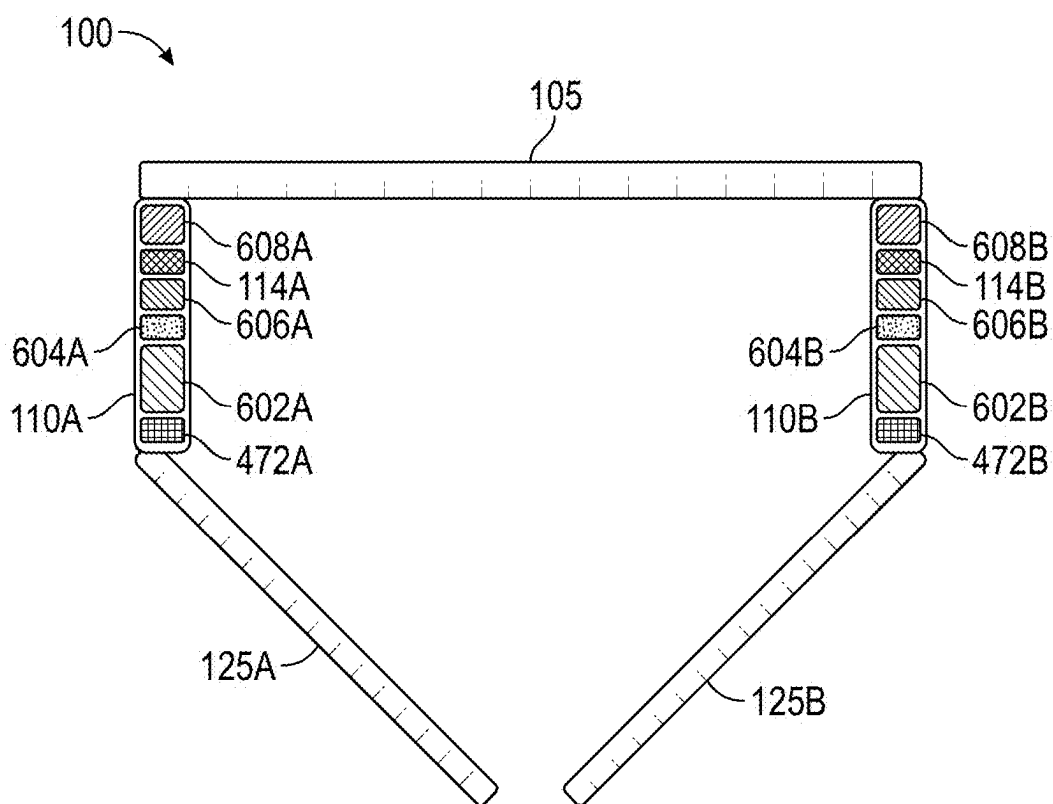


FIG. 6

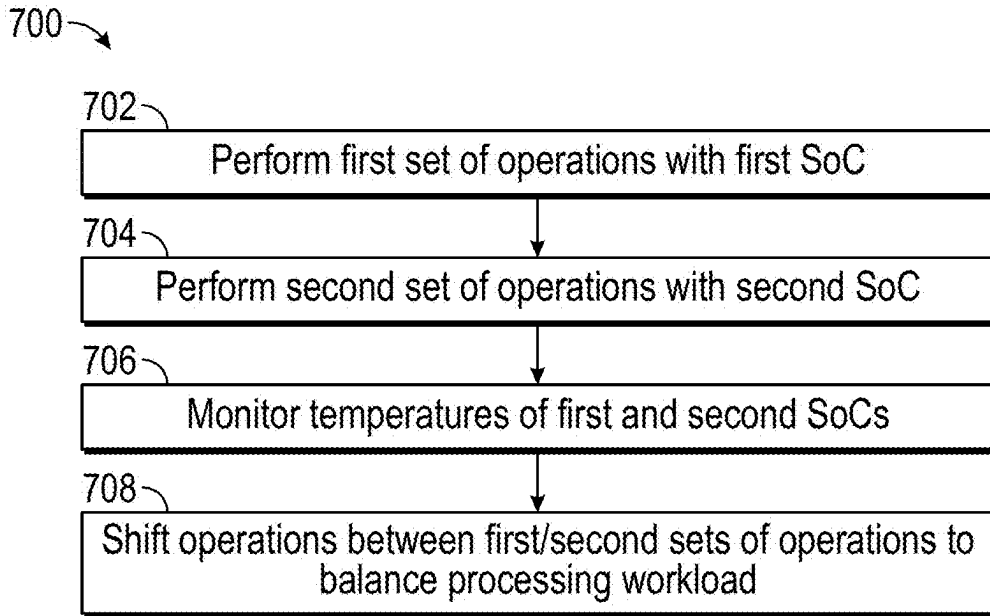


FIG. 7

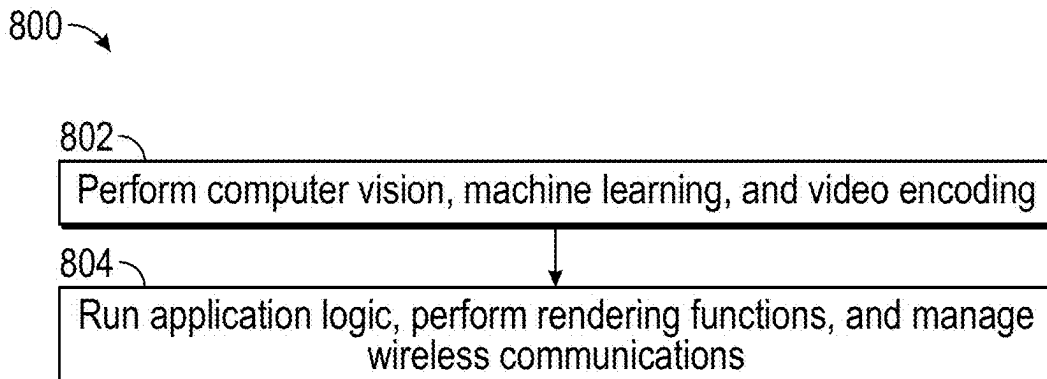


FIG. 8

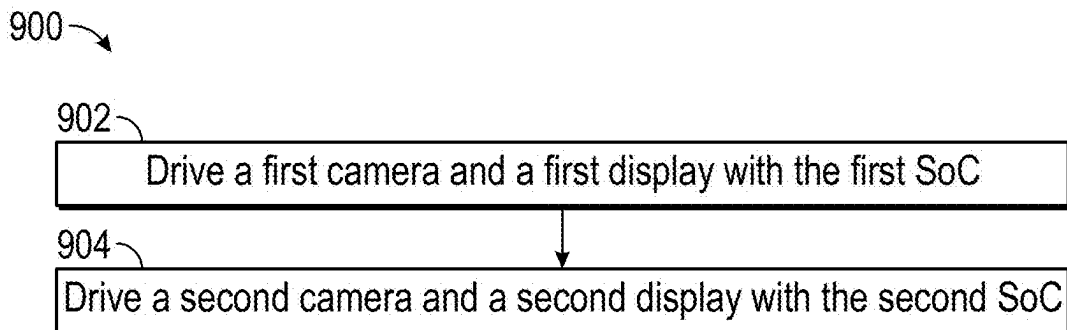


FIG. 9

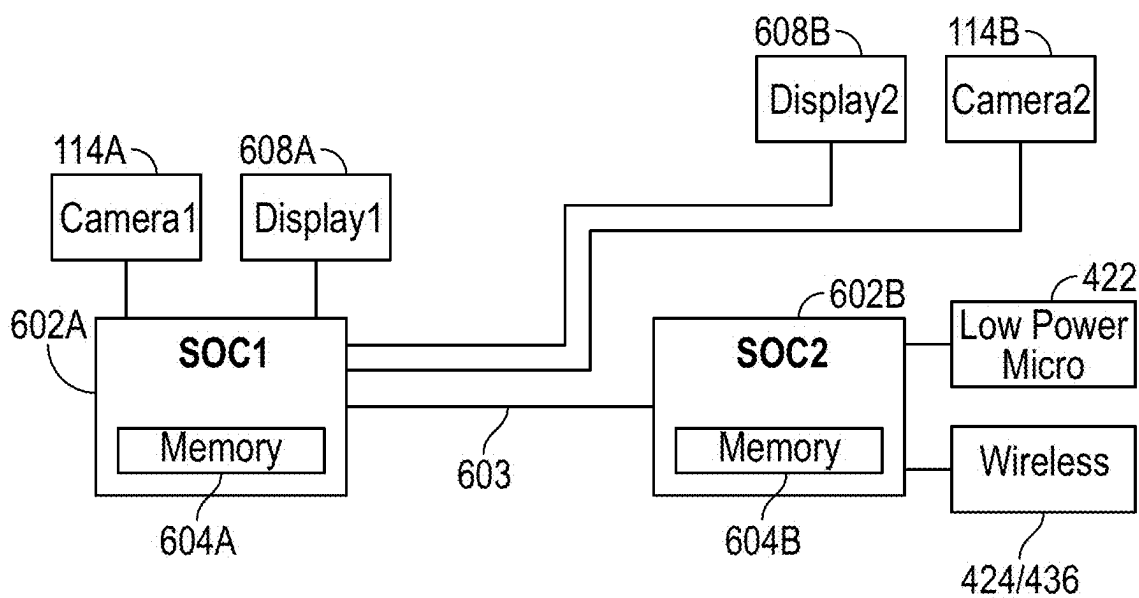


FIG. 10A

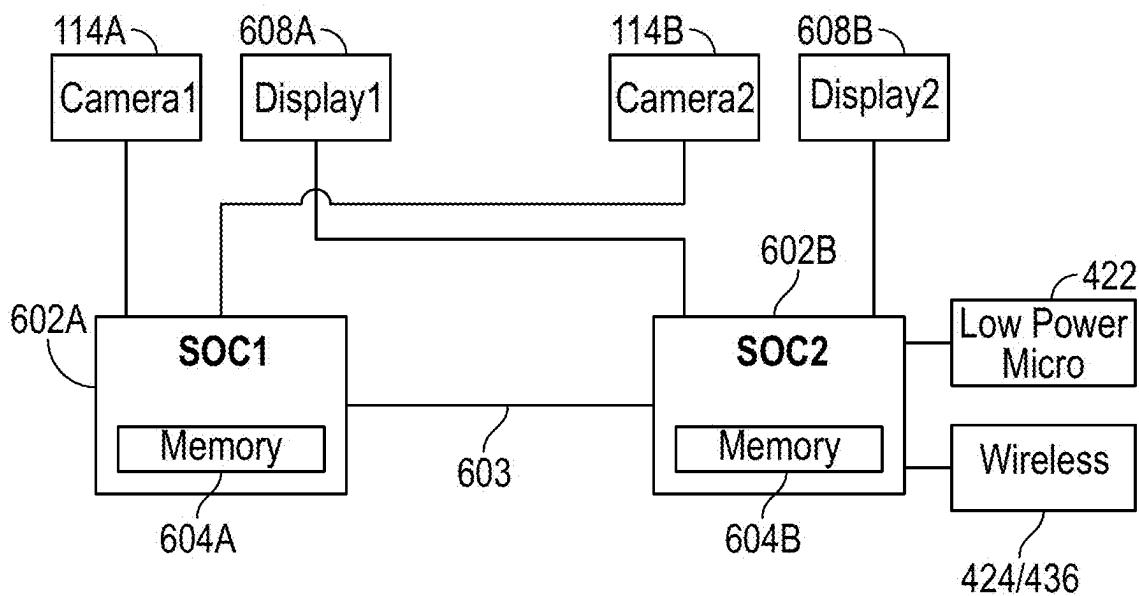


FIG. 10B

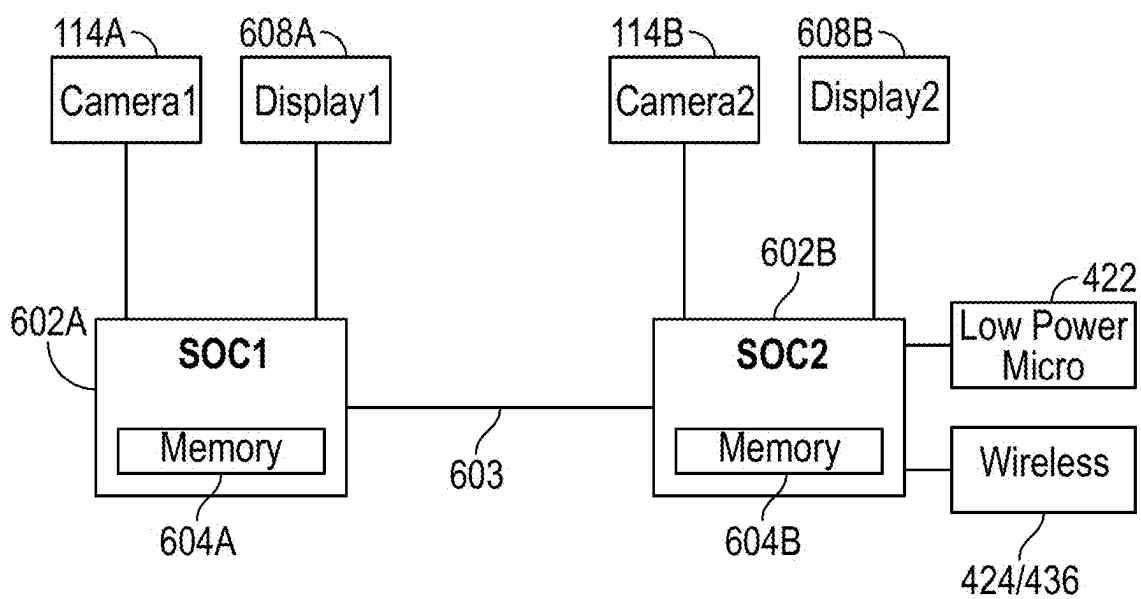


FIG. 10C

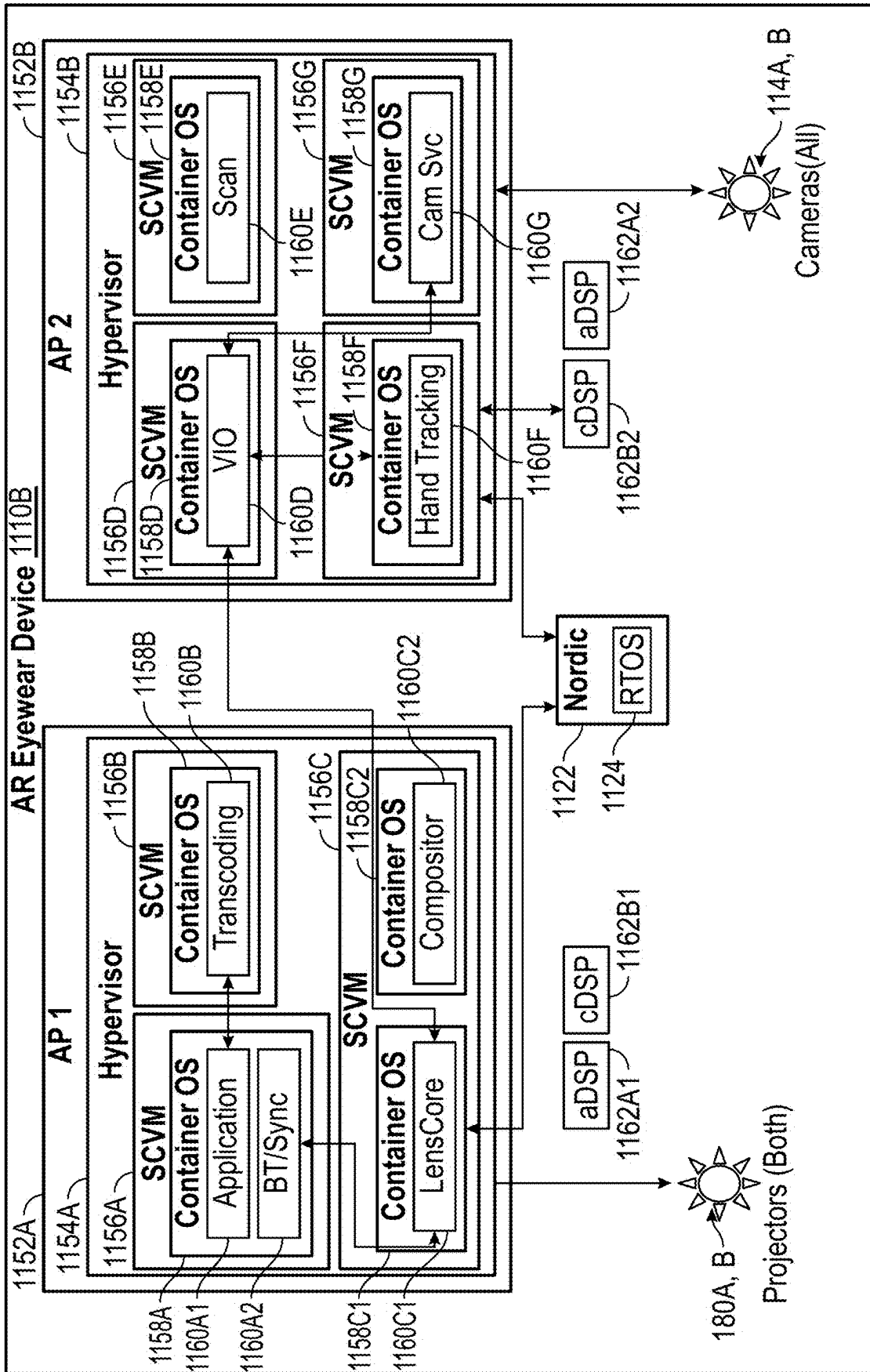


FIG. 11

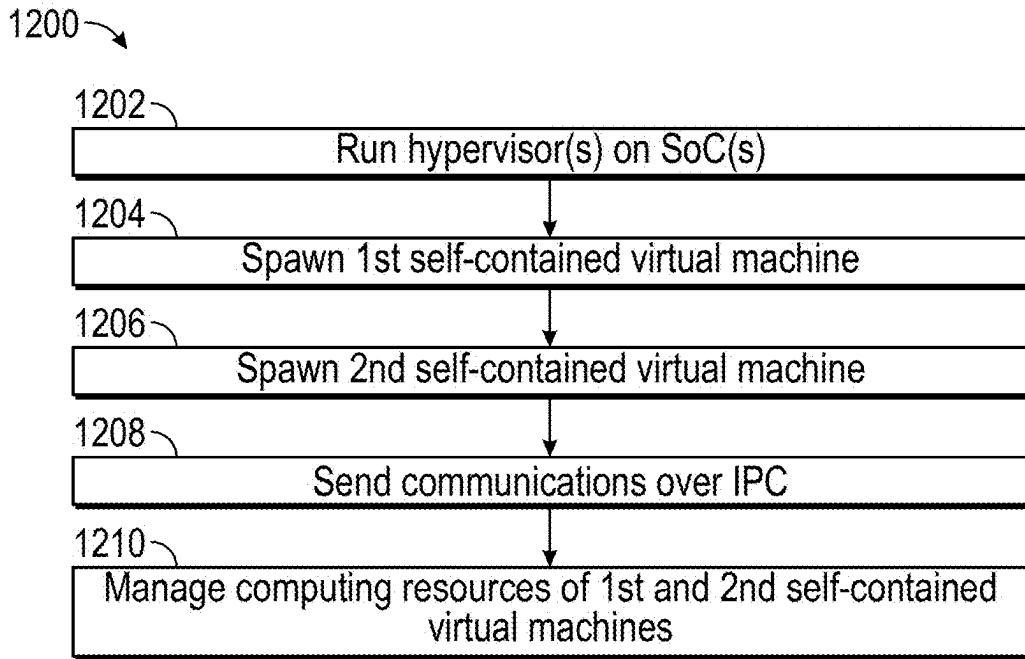


FIG. 12A

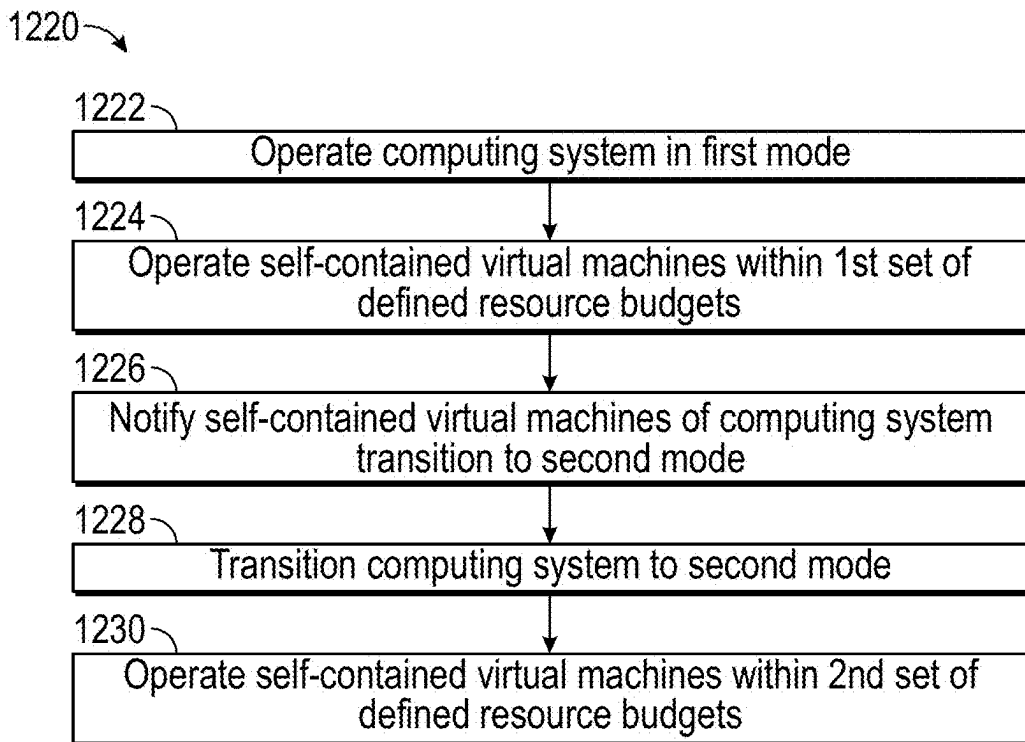


FIG. 12B

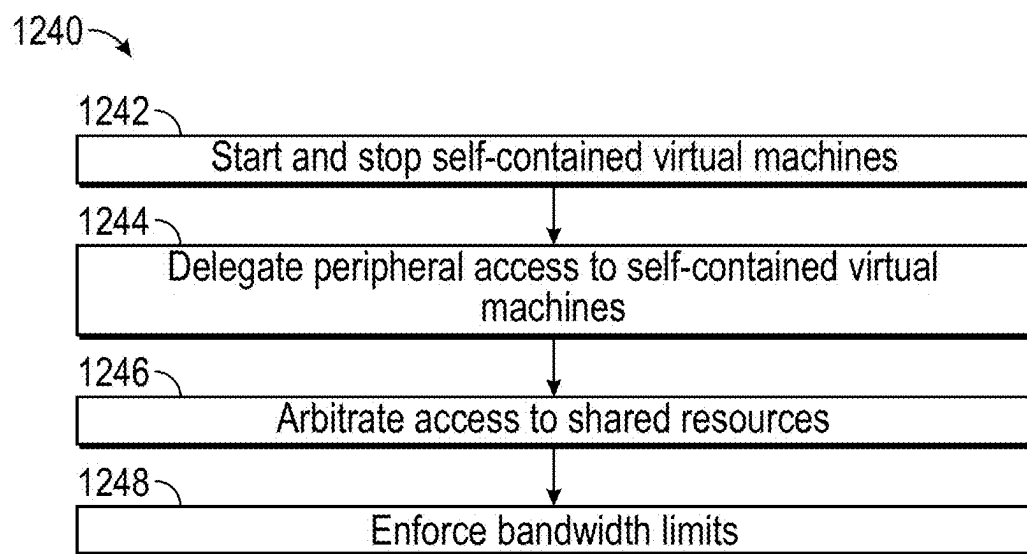


FIG. 12C

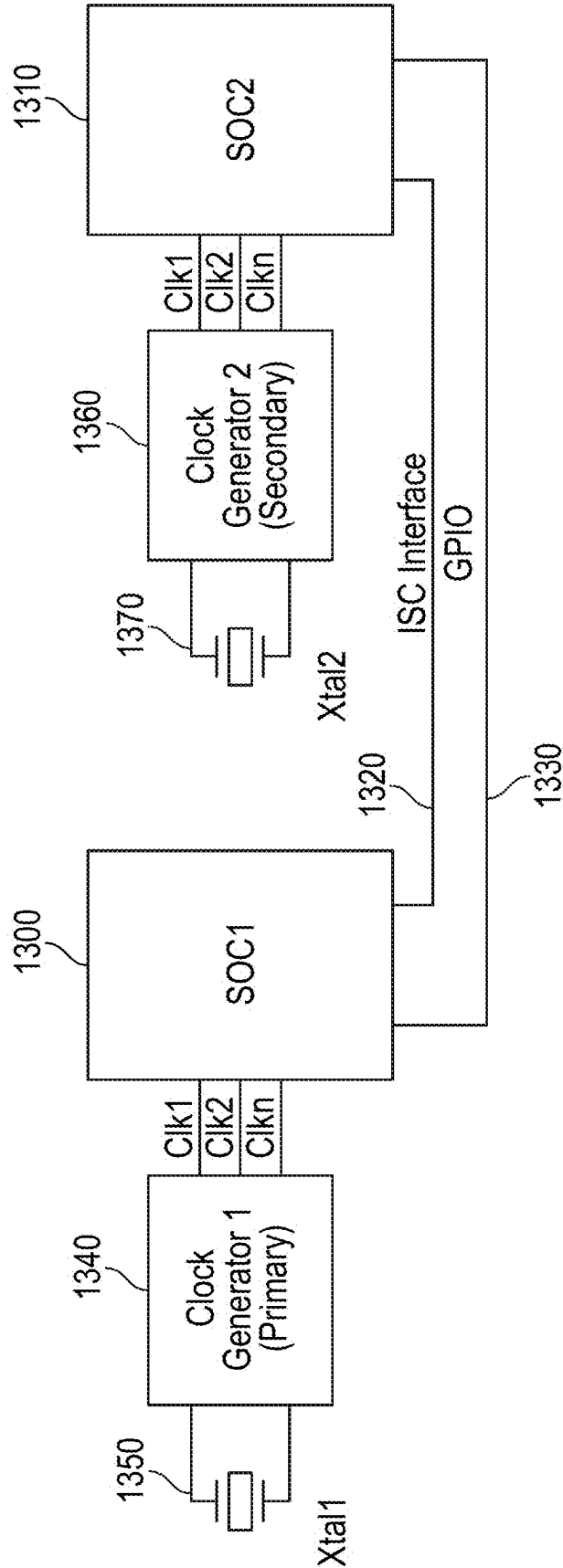


FIG. 13

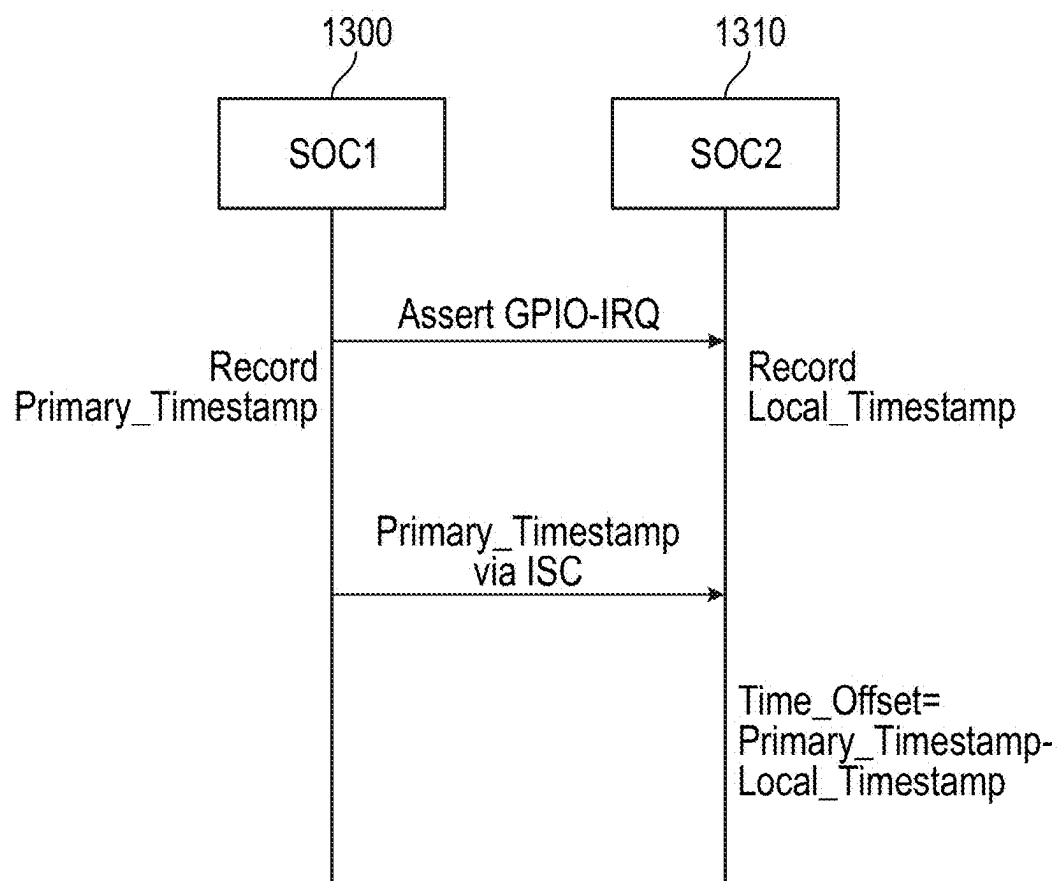


FIG. 14

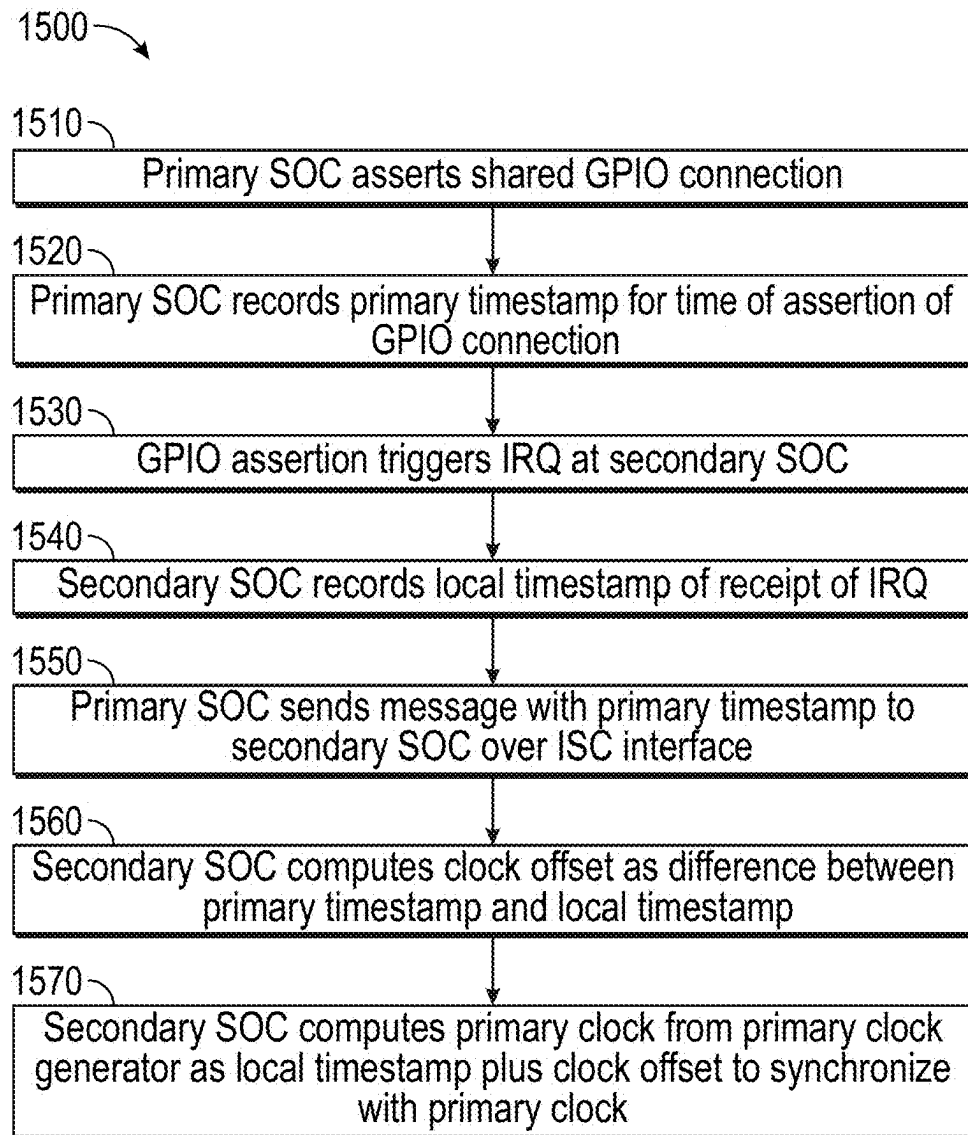


FIG. 15

SYNCHRONIZING SYSTEMS-ON-CHIP USING GPIO TIMESTAMPS

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is a Continuation of U.S. application Ser. No. 18/644,932 filed on Apr. 24, 2024, which is a Continuation of U.S. application Ser. No. 17/496,261 filed on Oct. 7, 2021, now U.S. Pat. No. 12,021,611, the contents of all of which are incorporated fully herein by reference.

TECHNICAL FIELD

[0002] The present subject matter relates to systems having multiple systems-on-chip (SoCs) and, more particularly, to techniques for synchronizing the respective SoCs by sharing timestamps recorded upon assertion of a general purpose input/output (GPIO) connection between the SoCs.

BACKGROUND

[0003] Many types of electronic devices available today, such as mobile devices (e.g., smartphones, tablets, and laptops), handheld devices, and wearable devices (e.g., smart glasses, digital eyewear, headwear, headgear, and head-mounted displays), include an operating system that supports a variety of cameras, sensors, wireless transceivers, input systems (e.g., touch-sensitive surfaces, pointers), peripheral devices, displays, and graphical user interfaces (GUIs) through which a user can interact with displayed content. Such electronic devices may include one or more SoCs for implementing the device functionality.

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] The features of the various examples described will be readily understood from the following detailed description, in which reference is made to the figures. A reference numeral is used with each element in the description and throughout the several views of the drawings. When a plurality of similar elements is present, a single reference numeral may be assigned to like elements, with an added letter referring to a specific element. The letter may be dropped when referring to more than one of the elements or a non-specific one of the elements.

[0005] The various elements shown in the figures are not drawn to scale unless otherwise indicated. The dimensions of the various elements may be enlarged or reduced in the interest of clarity. The several figures depict one or more implementations and are presented by way of example only and should not be construed as limiting. Included in the drawing are the following figures:

[0006] FIG. 1A is a side view (right) of an example hardware configuration of an eyewear device suitable for use in an eyewear system;

[0007] FIG. 1B is a perspective, partly sectional view of a right temple portion of the eyewear device of FIG. 1A depicting a right visible-light camera, and a circuit board;

[0008] FIG. 1C is a side view (left) of an example hardware configuration of the eyewear device of FIG. 1A, which shows a left visible-light camera;

[0009] FIG. 1D is a perspective, partly sectional view of a left temple portion of the eyewear device of FIG. 1C depicting the left visible-light camera, and a circuit board;

[0010] FIGS. 2A and 2B are rear views of example hardware configurations of an eyewear device utilized in the eyewear system;

[0011] FIG. 2C illustrates detecting eye gaze direction;

[0012] FIG. 2D illustrates detecting eye position;

[0013] FIG. 3 is a diagrammatic depiction of a three-dimensional scene formed by a left raw image captured by a left visible-light camera and a right raw image captured by a right visible-light camera;

[0014] FIG. 4 is a functional block diagram of an example eyewear system including an eyewear device connected to a mobile device and a server system via various networks;

[0015] FIG. 5 is a diagrammatic representation of an example hardware configuration for a mobile device of the eyewear system of FIG. 4;

[0016] FIG. 6 is a partial block diagram of an eyewear device with a first system-on-chip (SoC) adjacent one temple and a second SoC adjacent the other temple;

[0017] FIG. 7 is a flowchart of example steps for performing operations on eyewear with a first SoC and a second SoC;

[0018] FIG. 8 is a flowchart of example steps for a method of balancing processing workloads on an eyewear device between a first SoC and a second SoC;

[0019] FIG. 9 is a flowchart of example steps for another method of balancing processing workloads on an eyewear device between a first SoC and a second SoC;

[0020] FIGS. 10A, 10B, and 10C depict three respective strategies for dividing processing workload between a first SoC and a second SoC;

[0021] FIG. 11 is a block diagram illustrating an augmented reality headset with a virtual machine operating system;

[0022] FIGS. 12A, 12B, and 12C are flowcharts of example steps performed by a virtual machine operating system;

[0023] FIG. 13 illustrates a sample configuration for two SoCs that share timestamps recorded upon assertion of a general purpose input/output (GPIO) connection for synchronization of the two SoCs;

[0024] FIG. 14 illustrates a sequence diagram of the messaging sequence for recording timestamps via a GPIO connection between SoCs in a sample configuration; and

[0025] FIG. 15 is a flowchart illustrating a method for synchronizing two or more SoCs in the sample configuration.

DETAILED DESCRIPTION

[0026] Examples described herein relate to techniques for synchronizing independent systems-on-chip (SoCs) by sharing timestamps recorded upon asserting a general purpose input/output (GPIO) connection between the SoCs. In a sample configuration, a primary SoC asserts a shared GPIO connection between the primary SoC and a secondary SoC and records the primary clock timestamp (gpio_primary_time) for the time that the GPIO connection was asserted. Asserting the GPIO connection triggers an interrupt request (IRQ) at the secondary SoC, which records its local clock timestamp (gpio_local_time) for the time that the IRQ was received. The primary SoC sends a message including the primary clock timestamp to the secondary SoC over an inter-SoC bus. The secondary SoC uses the primary clock timestamp and its local clock timestamp to compute the time offset between the primary clock and its local clock as

$\text{time_offset} = \text{gpio_primary_time} - \text{gpio_local_time}$. The secondary SoC then can compute the primary clock's time locally as $\text{primary_time} = \text{local_time} + \text{time_offset}$. This approach requires little firmware overhead to implement but does require a GPIO connection to be established between the SoCs.

[0027] The systems and methods described herein are described for use in connection with an electronic eyewear device comprising two or more SoCs that work together to implement the functionality of the electronic eyewear device. It will be appreciated that the techniques described herein also may be used with other devices with two or more SoCs as described herein. Thus, the descriptions provided herein are for explanatory purposes only and not to limit the described systems and methods to any particular device or device configuration.

[0028] In sample configurations, systems, methods, and computer-readable media are described herein for synchronizing first and second SoCs having independent time bases that are used in an electronic eyewear device. The electronic eyewear device includes the first and second SoCs connected by a shared GPIO connection and an inter-SOC interface. The first and second SoCs are synchronized to each other by the first SoC asserting the GPIO connection to the second SoC where assertion of the GPIO connection triggers an interrupt request (IRQ) at the second SoC. The first SoC records a first timestamp for assertion of the shared GPIO connection, and the second SoC records a second timestamp of the time of receipt of the IRQ. The first SoC then sends a message including the first timestamp to the second SoC over the inter-SOC interface. The second SoC calculates a clock offset between the first and second SoCs as a difference between the first and second timestamps. The second SoC may further compute a primary clock of the first SoC as a sum of the local timestamp and the calculated clock offset, and the time clock of the secondary SoC may be adjusted to the computed primary clock.

[0029] The following detailed description includes systems, methods, techniques, instruction sequences, and computing machine program products illustrative of examples set forth in the disclosure. Numerous details and examples are included for the purpose of providing a thorough understanding of the disclosed subject matter and its relevant teachings. Those skilled in the relevant art, however, may understand how to apply the relevant teachings without such details. Aspects of the disclosed subject matter are not limited to the specific devices, systems, and method described because the relevant teachings can be applied or practiced in a variety of ways. The terminology and nomenclature used herein is for the purpose of describing particular aspects only and is not intended to be limiting. In general, well-known instruction instances, protocols, structures, and techniques are not necessarily shown in detail.

[0030] The terms “system-on-chip” or “SoC” are used herein to refer to an integrated circuit (also known as a “chip”) that integrates components of an electronic system on a single substrate or microchip. These components include a central processing unit (CPU), a graphical processing unit (GPU), an image signal processor (ISP), a memory controller, a video decoder, and a system bus interface for connection to another SoC. The components of the SoC may additionally include, by way of non-limiting example, one or more of an interface for an inertial measurement unit (IMU; e.g., I2C, SPI, I3C, etc.), a video

encoder, a transceiver (TX/RX; e.g., WI-FI®, BLUETOOTH®, or a combination thereof), and digital, analog, mixed-signal, and radio frequency signal processing functions.

[0031] The terms “virtual machine” or “VM” are used herein to refer to a software representation of a computer. A VM may be implemented in hardware, software, or a combination thereof.

[0032] The terms “self-contained virtual machine” or “SCVM” are used herein to refer to a virtual machine having an OS that is configured to provide at least one service. In one example, the SCVM regulates the resources that it uses (e.g., according to a resource budget), with the electronic device on which it operates provisioned to have those resources available. An SCVM may have more than one set of resources (e.g., multiple resource budgets), with the SCVM selecting the set of resources responsive to, for example, the operating mode of an electronic device on which the SCVM is present. Where a SCVM provides more than one service, each service runs in a respective container of the SCVM. Each container runs in a respective partition of the SCVM with a kernel of the SCVM implementing isolation between the containers.

[0033] The term “system isolation manager” is used herein to refer to computer software, firmware, or hardware (or a combination thereof) that manages a collection of virtual machines, containers, or a combination thereof to support isolation and communication between virtual machines/containers. Where virtual machines are managed to support isolation, the system isolation manager may be a hypervisor. Where containers are managed to support isolation, the system isolation manager may be a container manager such as Docker available from Docker, Inc. of Palo Alto, California, USA.

[0034] The term “hypervisor” is used herein to refer to computer software, firmware, or hardware (or a combination thereof) that creates and runs virtual machines. A computing system (e.g., an SoC) on which a hypervisor runs one or more virtual machines may be referred to as a host machine and each virtual machine may be referred to as a guest machine. The hypervisor presents the OSs of the guest machines with a virtual operating platform and manages the execution of the guest OSs.

[0035] The terms “operating system” and “OS” are used herein to refer to software that supports basic functions of a computer (real or virtual; e.g., a virtual machine), such as scheduling tasks, executing applications, and controlling peripherals. In one example, a supervisor OS is implemented in the hypervisor and a respective OS is implemented in each of the SCVMs.

[0036] The terms “coupled” or “connected” as used herein refer to any logical, optical, physical, or electrical connection, including a link or the like by which the electrical or magnetic signals produced or supplied by one system element are imparted to another coupled or connected system element. Unless described otherwise, coupled or connected elements or devices are not necessarily directly connected to one another and may be separated by intermediate components, elements, or communication media, one or more of which may modify, manipulate, or carry the electrical signals. The term “on” means directly supported by an element or indirectly supported by the element through another element that is integrated into or supported by the element.

[0037] The term “proximal” is used to describe an item or part of an item that is situated near, adjacent, or next to an object or person; or that is closer relative to other parts of the item, which may be described as “distal.” For example, the end of an item nearest an object may be referred to as the proximal end, whereas the generally opposing end may be referred to as the distal end.

[0038] The orientations of the eyewear device, other mobile devices, associated components, and any other devices incorporating a camera, an inertial measurement unit, or both such as shown in any of the drawings, are given by way of example only, for illustration and discussion purposes. In operation, the eyewear device may be oriented in any other direction suitable to the particular application of the eyewear device; for example, up, down, sideways, or any other orientation. Also, to the extent used herein, any directional term, such as front, rear, inward, outward, toward, left, right, lateral, longitudinal, up, down, upper, lower, top, bottom, side, horizontal, vertical, and diagonal are used by way of example only, and are not limiting as to the direction or orientation of any camera or inertial measurement unit as constructed or as otherwise described herein.

[0039] Additional objects, advantages and novel features of the examples will be set forth in part in the following description, and in part will become apparent to those skilled in the art upon examination of the following and the accompanying drawings or may be learned by production or operation of the examples. The objects and advantages of the present subject matter may be realized and attained by means of the methodologies, instrumentalities and combinations particularly pointed out in the appended claims.

[0040] Reference now is made in detail to the examples illustrated in the accompanying drawings and discussed below. FIGS. 1-12 describe an electronic eyewear device having two or more SoCs in which the systems and methods described herein may be implemented in sample configurations. FIGS. 13-15 describe a sample configuration of a technique for synchronizing respective SoCs by sharing timestamps recorded upon assertion of a general purpose input/output (GPIO) connection between the SoCs.

[0041] FIG. 1A is a side view (right) of an example hardware configuration of an eyewear device **100** which includes a touch-sensitive input device or touchpad **181**. As shown, the touchpad **181** may have a boundary that is subtle and not easily seen; alternatively, the boundary may be plainly visible or include a raised or otherwise tactile edge that provides feedback to the user about the location and boundary of the touchpad **181**. In other implementations, the eyewear device **100** may include a touchpad on the left side. The surface of the touchpad **181** is configured to detect finger touches, taps, and gestures (e.g., moving touches) for use with a GUI displayed by the eyewear device, on an image display, to allow the user to navigate through and select menu options in an intuitive manner, which enhances and simplifies the user experience.

[0042] Detection of finger inputs on the touchpad **181** can enable several functions. For example, touching anywhere on the touchpad **181** may cause the GUI to display or highlight an item on the image display, which may be projected onto at least one of the optical assemblies **180A**, **180B**. Double tapping on the touchpad **181** may select an item or icon. Sliding or swiping a finger in a particular direction (e.g., from front to back, back to front, up to down, or down to up) may cause the items or icons to slide or scroll

in a particular direction; for example, to move to a next item, icon, video, image, page, or slide. Sliding the finger in another direction may slide or scroll in the opposite direction; for example, to move to a previous item, icon, video, image, page, or slide. The touchpad **181** can be virtually anywhere on the eyewear device **100**.

[0043] In one example, an identified finger gesture of a single tap on the touchpad **181**, initiates selection or pressing of a graphical user interface element in the image presented on the image display of the optical assembly **180A**, **180B**. An adjustment to the image presented on the image display of the optical assembly **180A**, **180B** based on the identified finger gesture can be a primary action which selects or submits the graphical user interface element on the image display of the optical assembly **180A**, **180B** for further display or execution.

[0044] As shown in FIG. 1A, the eyewear device **100** includes a right visible-light camera **114B**. As further described herein, two cameras **114A**, **114B** capture image information for a scene from two separate viewpoints. The two captured images may be used to project a three-dimensional display onto an image display for viewing on or with 3D glasses.

[0045] The eyewear device **100** includes a right optical assembly **180B** with an image display to present images, such as depth images. As shown in FIGS. 1A and 1B, the eyewear device **100** includes the right visible-light camera **114B**. The eyewear device **100** can include multiple visible-light cameras **114A**, **114B** that form a passive type of three-dimensional camera, such as stereo camera, of which the right visible-light camera **114B** is located on a right temple portion **110B**. As shown in FIGS. 1C-D, the eyewear device **100** also includes a left visible-light camera **114A** location on a left temple portion **110A**.

[0046] Left and right visible-light cameras **114A**, **114B** are sensitive to the visible-light range wavelength. Each of the visible-light cameras **114A**, **114B** have a different frontward facing field of view which are overlapping to enable generation of three-dimensional depth images. Right visible-light camera **114B** captures a right field of view **111B** and left visible-light camera **114A** captures a left field of view **111A**. Generally, a “field of view” is the part of the scene that is visible through the camera at a particular position and orientation in space. The fields of view **111A** and **111B** have an overlapping field of view **304** (FIG. 3). Objects or object features outside the field of view **111A**, **111B** when the visible-light camera captures the image are not recorded in a raw image (e.g., photograph or picture). The field of view describes an angle range or extent in which the image sensor of the visible-light camera **114A**, **114B** picks up electromagnetic radiation of a given scene in a captured image of the given scene. Field of view can be expressed as the angular size of the view cone; i.e., an angle of view. The angle of view can be measured horizontally, vertically, or diagonally.

[0047] In an example, visible-light cameras **114A**, **114B** have a field of view with an angle of view between 15° to 30°, for example 24°, and have a resolution of 480x480 pixels or greater. In another example, the field of view can be much wider, such as 100° or greater. The “angle of coverage” describes the angle range that a lens of visible-light cameras **114A**, **114B** or infrared camera **220** (see FIG. 2A) can effectively image. Typically, the camera lens produces an image circle that is large enough to cover the film or sensor of the camera completely, possibly including some

vignetting (e.g., a darkening of the image toward the edges when compared to the center). If the angle of coverage of the camera lens does not fill the sensor, the image circle will be visible, typically with strong vignetting toward the edge, and the effective angle of view will be limited to the angle of coverage.

[0048] Examples of such visible-light cameras **114A**, **114B** include a high-resolution complementary metal-oxide-semiconductor (CMOS) image sensor and a digital VGA camera (video graphics array) capable of resolutions of 640p (e.g., 640×480 pixels for a total of 0.3 megapixels), 720p, or 1080p. Other examples of visible-light cameras **114A**, **114B** that can capture high-definition (HD) still images and store them at a resolution of 1642 by 1642 pixels (or greater); or record high-definition video at a high frame rate (e.g., thirty to sixty frames per second or more) and store the recording at a resolution of 1216 by 1216 pixels (or greater).

[0049] The eyewear device **100** may capture image sensor data from the visible-light cameras **114A**, **114B** along with geolocation data, digitized by an image processor, for storage in a memory. The visible-light cameras **114A**, **114B** capture respective left and right raw images in the two-dimensional space domain that comprise a matrix of pixels on a two-dimensional coordinate system that includes an X-axis for horizontal position and a Y-axis for vertical position. Each pixel includes a color attribute value (e.g., a red pixel light value, a green pixel light value, or a blue pixel light value) and a position attribute (e.g., an X-axis coordinate and a Y-axis coordinate).

[0050] In order to capture stereo images for later display as a three-dimensional projection, an image processor **412** (shown in FIG. 4) may be coupled to the visible-light cameras **114A**, **114B** to receive and store the visual image information. The image processor **412**, or another processor, controls operation of the visible-light cameras **114A**, **114B** to act as a stereo camera simulating human binocular vision and may add a timestamp to each image. The timestamp on each pair of images allows display of the images together as part of a three-dimensional projection. Three-dimensional projections produce an immersive, life-like experience that is desirable in a variety of contexts, including virtual reality (VR) and video gaming.

[0051] FIG. 1B is a perspective, cross-sectional view of a right temple portion **110B** of the eyewear device **100** of FIG. 1A depicting the right visible-light camera **114B** of the camera system, and a circuit board. FIG. 1C is a side view (left) of an example hardware configuration of an eyewear device **100** of FIG. 1A, which shows a left visible-light camera **114A** of the camera system. FIG. 1D is a perspective, cross-sectional view of a left temple portion **110A** of the eyewear device of FIG. 1C depicting the left visible-light camera **114A** of the three-dimensional camera, and a circuit board. Construction and placement of the left visible-light camera **114A** is substantially similar to the right visible-light camera **114B**, except the connections and coupling are on the left lateral side **170A**.

[0052] As shown in the example of FIG. 1B, the eyewear device **100** includes the right visible-light camera **114B** and a circuit board **140B**, which may be a flexible printed circuit board (PCB). A right hinge **126B** connects the right temple portion **110B** to a right temple **125B** of the eyewear device **100**. In some examples, components of the right visible-light camera **114B**, the flexible PCB **140B**, or other electrical connectors or contacts may be located on the right temple

125B, the right hinge **126B**, the right temple portion **110B**, the frame **105**, or a combination thereof. The components (or subset thereof) may be incorporated in a SoC.

[0053] As shown in the example of FIG. 1D, the eyewear device **100** includes the left visible-light camera **114A** and a circuit board **140A**, which may be a flexible printed circuit board (PCB). A left hinge **126A** connects the left temple portion **110A** to a left temple **125A** of the eyewear device **100**. In some examples, components of the left visible-light camera **114A**, the flexible PCB **140A**, or other electrical connectors or contacts may be located on the left temple **125A**, the left hinge **126A**, the left temple portion **110A**, the frame **105**, or a combination thereof. The components (or subset thereof) may be incorporated in a SoC.

[0054] The left temple portion **110A** and the right temple portion **110B** includes temple portion body **190** and a temple portion cap, with the temple portion cap omitted in the cross-section of FIG. 1B and FIG. 1D. Disposed inside the left temple portion **110A** and the right temple portion **110B** are various interconnected circuit boards, such as PCBs or flexible PCBs, that include controller circuits for the respective left visible-light camera **114A** and the right visible-light camera **114B**, microphone(s) **130**, speaker **132**, low-power wireless circuitry (e.g., for wireless short range network communication via BLUETOOTH®), high-speed wireless circuitry (e.g., for wireless local area network communication via WI-FI®). The components and circuitry (or subset thereof) in each temple portion **110** may be incorporated in a SoC.

[0055] The right visible-light camera **114B** is coupled to or disposed on the flexible PCB **140B** and covered by a visible-light camera cover lens, which is aimed through opening(s) formed in the frame **105**. For example, the right rim **107B** of the frame **105**, shown in FIG. 2A, is connected to the right temple portion **110B** and includes the opening(s) for the visible-light camera cover lens. The frame **105** includes a front side configured to face outward and away from the eye of the user. The opening for the visible-light camera cover lens is formed on and through the front or outward-facing side of the frame **105**. In the example, the right visible-light camera **114B** has an outward-facing field of view **111B** (shown in FIG. 3) with a line of sight or perspective that is correlated with the right eye of the user of the eyewear device **100**. The visible-light camera cover lens can also be adhered to a front side or outward-facing surface of the right temple portion **110B** in which an opening is formed with an outward-facing angle of coverage, but in a different outwardly direction. The coupling can also be indirect via intervening components. Although shown as being formed on the circuit boards of the right temple portion **110B**, the right visible-light camera **114B** can be formed on the circuit boards of the left temple **125B** or the frame **105**.

[0056] The left visible-light camera **114A** is coupled to or disposed on the flexible PCB **140A** and covered by a visible-light camera cover lens, which is aimed through opening(s) formed in the frame **105**. For example, the left rim **107A** of the frame **105**, shown in FIG. 2A, is connected to the left temple portion **110A** and includes the opening(s) for the visible-light camera cover lens. The frame **105** includes a front side configured to face outward and away from the eye of the user. The opening for the visible-light camera cover lens is formed on and through the front or outward-facing side of the frame **105**. In the example, the

left visible-light camera **114A** has an outward-facing field of view **111A** (shown in FIG. 3) with a line of sight or perspective that is correlated with the left eye of the user of the eyewear device **100**. The visible-light camera cover lens can also be adhered to a front side or outward-facing surface of the left temple portion **110A** in which an opening is formed with an outward-facing angle of coverage, but in a different outwardly direction. The coupling can also be indirect via intervening components. Although shown as being formed on the circuit boards of the left temple portion **110A**, the left visible-light camera **114A** can be formed on the circuit boards of the left temple **125A** or the frame **105**.

[0057] FIGS. 2A and 2B are perspective views, from the rear, of example hardware configurations of the eyewear device **100**, including two different types of image displays. The eyewear device **100** is sized and shaped in a form configured for wearing by a user; the form of eyeglasses is shown in the example. The eyewear device **100** can take other forms and may incorporate other types of frameworks; for example, a headgear, a headset, or a helmet.

[0058] In the eyeglasses example, eyewear device **100** includes a frame **105** including a left rim **107A** connected to a right rim **107B** via a bridge **106** adapted to be supported by a nose of the user. The left and right rims **107A**, **107B** include respective apertures **175A**, **175B**, which hold a respective optical element **180A**, **180B**, such as a lens and a display device. As used herein, the term “lens” is meant to include transparent or translucent pieces of glass or plastic having curved or flat surfaces that cause light to converge/diverge or that cause little or no convergence or divergence.

[0059] Although shown as having two optical elements **180A**, **180B**, the eyewear device **100** can include other arrangements, such as a single optical element (or it may not include any optical element **180A**, **180B**), depending on the application or the intended user of the eyewear device **100**. As further shown, eyewear device **100** includes a left temple portion **110A** adjacent the left lateral side **170A** of the frame **105** and a right temple portion **110B** adjacent the right lateral side **170B** of the frame **105**. The temple portions **110A**, **110B** may be integrated into the frame **105** on the respective lateral sides **170A**, **170B** (as illustrated) or implemented as separate components attached to the frame **105** on the respective lateral sides **170A**, **170B**. Alternatively, the temple portions **110A**, **110B** may be integrated into temples (not shown) attached to the frame **105**.

[0060] In one example, the image display of optical assembly **180A**, **180B** includes an integrated image display **177**. As shown in FIG. 2A, each optical assembly **180A**, **180B** includes a suitable display matrix **177**, such as a liquid crystal display (LCD), an organic light-emitting diode (OLED) display, or any other such display. Each optical assembly **180A**, **180B** also includes an optical layer or layers **176**, which can include lenses, optical coatings, prisms, mirrors, waveguides, optical strips, and other optical components in any combination. The optical layers **176A**, **176B**, . . . **176N** (shown as **176A-N** in FIG. 2A and herein) can include a prism having a suitable size and configuration and including a first surface for receiving light from a display matrix and a second surface for emitting light to the eye of the user. The prism of the optical layers **176A-N** extends over all or at least a portion of the respective apertures **175A**, **175B** formed in the left and right rims **107A**, **107B** to permit the user to see the second surface of the prism when the eye of the user is viewing through the corresponding left and

right rims **107A**, **107B**. The first surface of the prism of the optical layers **176A-N** faces upwardly from the frame **105** and the display matrix **177** overlies the prism so that photons and light emitted by the display matrix **177** impinge the first surface. The prism is sized and shaped so that the light is refracted within the prism and is directed toward the eye of the user by the second surface of the prism of the optical layers **176A-N**. In this regard, the second surface of the prism of the optical layers **176A-N** can be convex to direct the light toward the center of the eye. The prism can optionally be sized and shaped to magnify the image projected by the display matrix **177**, and the light travels through the prism so that the image viewed from the second surface is larger in one or more dimensions than the image emitted from the display matrix **177**.

[0061] In one example, the optical layers **176A-N** may include an LCD layer that is transparent (keeping the lens open) unless and until a voltage is applied which makes the layer opaque (closing or blocking the lens). The image processor **412** on the eyewear device **100** may execute programming to apply the voltage to the LCD layer in order to produce an active shutter system, making the eyewear device **100** suitable for viewing visual content when displayed as a three-dimensional projection. Technologies other than LCD may be used for the active shutter mode, including other types of reactive layers that are responsive to a voltage or another type of input.

[0062] In another example, the image display device of optical assembly **180A**, **180B** includes a projection image display as shown in FIG. 2B. Each optical assembly **180A**, **180B** may include a laser projector **150**, which is a three-color laser projector using a scanning mirror or galvanometer. During operation, an optical source such as laser projector **150** is disposed in or on one or both of the temples **125A**, **125B** of the eyewear device **100**. Optical assembly **180B** in this example includes one or more optical strips **155A**, **155B**, . . . **155N** (shown as **155A-N** in FIG. 2B) which are spaced apart and across the width of the lens of each optical assembly **180A**, **180B** or across a depth of the lens between the front surface and the rear surface of the lens.

[0063] As the photons projected by the laser projector **150** travel across the lens of each optical assembly **180A**, **180B**, the photons encounter the optical strips **155A-N**. When a particular photon encounters a particular optical strip, the photon is either redirected toward the user's eye, or it passes to the next optical strip. A combination of modulation of laser projector **150**, and modulation of optical strips, may control specific photons or beams of light. In an example, a processor controls optical strips **155A-N** by initiating mechanical, acoustic, or electromagnetic signals. Although shown as having two optical assemblies **180A**, **180B**, the eyewear device **100** can include other arrangements, such as a single or three optical assemblies, or each optical assembly **180A**, **180B** may have arranged different arrangement depending on the application or intended user of the eyewear device **100**.

[0064] In another example, the eyewear device **100** shown in FIG. 2B may include two projectors, a left projector (not shown) and a right projector (shown as projector **150**). The left optical assembly **180A** may include a left display matrix (not shown) or a left set of optical strips (not shown) which are configured to interact with light from the left projector. In this example, the eyewear device **100** includes a left display and a right display.

[0065] As further shown in FIGS. 2A and 2B, eyewear device 100 includes a left temple portion 110A adjacent the left lateral side 170A of the frame 105 and a right temple portion 110B adjacent the right lateral side 170B of the frame 105. The temple portions 110A, 110B may be integrated into the frame 105 on the respective lateral sides 170A, 170B (as illustrated) or implemented as separate components attached to the frame 105 on the respective lateral sides 170A, 170B. Alternatively, the temple portions 110A, 110B may be integrated into temples 125A, 125B attached to the frame 105.

[0066] As shown in FIG. 2A, the frame 105 or one or more of the left and right temples 110A-B may include an infrared emitter 215 and an infrared camera 220. The infrared emitter 215 and the infrared camera 220 can be connected to the flexible PCB 140B by soldering, for example. Other arrangements of the infrared emitter 215 and infrared camera 220 can be implemented, including arrangements in which the infrared emitter 215 and infrared camera 220 are both on the right rim 107B, or in different locations on the frame 105, for example, the infrared emitter 215 is on the left rim 107A and the infrared camera 220 is on the right rim 107B. In another example, the infrared emitter 215 is on the frame 105 and the infrared camera 220 is on one of the temples 110A-B, or vice versa. The infrared emitter 215 can be connected essentially anywhere on the frame 105, left temple 110A, or right temple 110B to emit a pattern of infrared light. Similarly, the infrared camera 220 can be connected essentially anywhere on the frame 105, left temple 110A, or right temple 110B to capture at least one reflection variation in the emitted pattern of infrared light.

[0067] The infrared emitter 215 and infrared camera 220 are arranged to face inwards towards an eye of the user with a partial or full field of view of the eye in order to identify the respective eye position and gaze direction. For example, the infrared emitter 215 and infrared camera 220 are positioned directly in front of the eye, in the upper part of the frame 105 or in the temples 110A-B at either end of the frame 105.

[0068] In an example, the processor 432 (FIG. 4) utilizes an eye tracker 213 to determine an eye gaze direction 230 of a wearer's eye 234 as shown in FIG. 2C, and an eye position 236 of the wearer's eye 234 within an eyebox as shown in FIG. 2D. In one example, the eye tracker 213 is a scanner which uses infrared light illumination (e.g., near-infrared, short-wavelength infrared, mid-wavelength infrared, long-wavelength infrared, or far infrared) to capture images of reflection variations of infrared light from the eye 234 to determine the gaze direction 230 of a pupil 232 of the eye 234, and also the eye position 236 with respect to the display 180D, which may include one or both of optical assemblies 180A and 180B.

[0069] FIG. 3 is a diagrammatic depiction of a three-dimensional scene 306, a left raw image 302A captured by a left visible-light camera 114A, and a right raw image 302B captured by a right visible-light camera 114B. The left field of view 111A may overlap, as shown, with the right field of view 111B. The overlapping field of view 304 represents that portion of the image captured by both cameras 114A, 114B. The term 'overlapping' when referring to field of view means the matrix of pixels in the generated raw images overlap by thirty percent (30%) or more. 'Substantially overlapping' means the matrix of pixels in the generated raw images—or in the infrared image of scene—overlap by fifty

percent (50%) or more. As described herein, the two raw images 302A, 302B may be processed to include a time-stamp, which allows the images to be displayed together as part of a three-dimensional projection.

[0070] For the capture of stereo images, as illustrated in FIG. 3, a pair of raw red, green, and blue (RGB) images are captured of a real scene 306 at a given moment in time—a left raw image 302A captured by the left camera 114A and right raw image 302B captured by the right camera 114B. When the pair of raw images 302A, 302B are processed (e.g., by the image processor 412), depth images are generated. The generated depth images may be viewed on an optical assembly 180A, 180B of an eyewear device, on another display (e.g., the image display 580 on a mobile device 401), or on a screen.

[0071] The generated depth images are in the three-dimensional space domain and can comprise a matrix of vertices on a three-dimensional location coordinate system that includes an X axis for horizontal position (e.g., length), a Y axis for vertical position (e.g., height), and a Z axis for depth (e.g., distance). Each vertex may include a color attribute (e.g., a red pixel light value, a green pixel light value, or a blue pixel light value); a position attribute (e.g., an X location coordinate, a Y location coordinate, and a Z location coordinate); a texture attribute; a reflectance attribute; or a combination thereof. The texture attribute quantifies the perceived texture of the depth image, such as the spatial arrangement of color or intensities in a region of vertices of the depth image.

[0072] In one example, an eyewear system 400 (FIG. 4) includes the eyewear device 100, which includes a frame 105, a left temple 110A extending from a left lateral side 170A of the frame 105, and a right temple 125B extending from a right lateral side 170B of the frame 105. The eyewear device 100 may further include at least two visible-light cameras 114A, 114B having overlapping fields of view. In one example, the eyewear device 100 includes a left visible-light camera 114A with a left field of view 111A, as illustrated in FIG. 3. The left camera 114A is connected to the frame 105, left temple 125A, or left temple portion 110A to capture a left raw image 302A from the left side of scene 306. The eyewear device 100 further includes a right visible-light camera 114B with a right field of view 111B. The right camera 114B is connected to the frame 105, right temple 125B, or right temple portion 110B to capture a right raw image 302B from the right side of scene 306.

[0073] FIG. 4 is a functional block diagram of an example eyewear system 400 that includes a wearable device (e.g., an eyewear device 100), a mobile device 401, and a server system 498 connected via various networks 495 such as the Internet. The eyewear system 400 includes a low-power wireless connection 425 and a high-speed wireless connection 437 between the eyewear device 100 and the mobile device 401.

[0074] As shown in FIG. 4, the eyewear device 100 includes one or more visible-light cameras 114A, 114B that capture still images, video images, or both still and video images, as described herein. The cameras 114A, 114B may have a direct memory access (DMA) to high-speed circuitry 430 and function as a stereo camera. The cameras 114A, 114B may be used to capture initial-depth images that may be rendered into three-dimensional (3D) models that are texture-mapped images of a red, green, and blue (RGB) imaged scene.

[0075] The eyewear device 100 further includes two optical assemblies 180A, 180B (one associated with the left lateral side 170A and one associated with the right lateral side 170B). The eyewear device 100 also includes an image display driver 442, an image processor 412, low-power circuitry 420, and high-speed circuitry 430 (all of which may be duplicated and incorporated into a pair of SoCs located on the eyewear device 100). The image displays 177 of each optical assembly 180A, 180B are for presenting images, including still images, video images, or still and video images. The image display driver 442 is coupled to the image displays of each optical assembly 180A, 180B in order to control the display of images.

[0076] The eyewear device 100 additionally includes one or more microphones 130 and speakers 132 (e.g., one of each associated with the left side of the eyewear device and another associated with the right side of the eyewear device). The microphones 130 and speakers 132 may be incorporated into the frame 105, temples 125, or temple portions 110 of the eyewear device 100. The one or more speakers 132 are driven by audio processor 443 (which may be duplicated and incorporated into a pair of SoCs) under control of low-power circuitry 420, high-speed circuitry 430, or both. The speakers 132 are for presenting audio signals including, for example, a beat track. The audio processor 443 is coupled to the speakers 132 in order to control the presentation of sound.

[0077] The components shown in FIG. 4 for the eyewear device 100 are located on one or more circuit boards, for example a printed circuit board (PCB) or flexible printed circuit (FPC) of one or more SoCs, located in the rims or temples. Alternatively, or additionally, the depicted components can be located in the temple portions, frames, hinges, or bridge of the eyewear device 100. Left and right visible-light cameras 114A, 114B can include digital camera elements such as a complementary metal-oxide-semiconductor (CMOS) image sensor, a charge-coupled device, a lens, or any other respective visible or light capturing elements that may be used to capture data, including still images or video of scenes with unknown objects.

[0078] As shown in FIG. 4, high-speed circuitry 430 includes a high-speed processor 432, a memory 434, and high-speed wireless circuitry 436. In the example, the image display driver 442 is coupled to the high-speed circuitry 430 and operated by the high-speed processor 432 in order to drive the left and right image displays of each optical assembly 180A, 180B. High-speed processor 432 may be any processor capable of managing high-speed communications and operation of any general computing system needed for eyewear device 100. High-speed processor 432 includes processing resources needed for managing high-speed data transfers on high-speed wireless connection 437 to a wireless local area network (WLAN) using high-speed wireless circuitry 436.

[0079] In some examples, the high-speed processor 432 executes an OS such as a LINUX OS or other such OS of the eyewear device 100 and the OS is stored in memory 434 for execution. In addition to any other responsibilities, the high-speed processor 432 executes a software architecture for the eyewear device 100 that is used to manage data transfers with high-speed wireless circuitry 436. In some examples, high-speed wireless circuitry 436 is configured to implement Institute of Electrical and Electronic Engineers (IEEE) 802.11 communication standards, also referred to

herein as WI-FI®. In other examples, other high-speed communications standards may be implemented by high-speed wireless circuitry 436.

[0080] The low-power circuitry 420 includes a low-power processor 422 and low-power wireless circuitry 424. The low-power wireless circuitry 424 and the high-speed wireless circuitry 436 of the eyewear device 100 can include short-range transceivers (BLUETOOTH® or Bluetooth Low-Energy (BLE)) and wireless wide, local, or wide-area network transceivers (e.g., cellular or WI-FI®). Mobile device 401, including the transceivers communicating via the low-power wireless connection 425 and the high-speed wireless connection 437, may be implemented using details of the architecture of the eyewear device 100, as can other elements of the network 495.

[0081] Memory 434 includes any storage device capable of storing various data and applications, including, among other things, camera data generated by the left and right visible-light cameras 114A, 114B, the infrared camera(s) 220 in response to reflected emissions from infrared emitter 215, the image processor 412, and images generated for display 177 by the image display driver 442 on the image display of each optical assembly 180A, 180B. Although the memory 434 is shown as integrated with high-speed circuitry 430, the memory 434 in other examples may be an independent, standalone element of the eyewear device 100. In certain such examples, electrical routing lines may provide a connection through a chip that includes the high-speed processor 432 from the image processor 412 or low-power processor 422 to the memory 434. In other examples, the high-speed processor 432 may manage addressing of memory 434 such that the low-power processor 422 will boot the high-speed processor 432 any time that a read or write operation involving memory 434 is needed.

[0082] As shown in FIG. 4, the high-speed processor 432 of the eyewear device 100 can be coupled to the camera system (visible-light cameras 114A, 114B), the image display driver 442, the user input device 491, and the memory 434. As shown in FIG. 5, the CPU 530 of the mobile device 401 may be coupled to a camera system 570, a mobile display driver 582, a user input layer 591, and a flash memory 540A.

[0083] The server system 498 may be one or more computing devices as part of a service or network computing system, for example, that include a processor, a memory, and network communication interface to communicate over the network 495 with one or more eyewear devices 100 and 100B and a mobile device 401.

[0084] The output components of the eyewear device 100 include visual elements, such as the left and right image displays associated with each lens or optical assembly 180A, 180B as described in FIGS. 2A and 2B (e.g., a display such as a liquid crystal display (LCD), a plasma display panel (PDP), a light emitting diode (LED) display, a projector, or a waveguide). The eyewear device 100 may include a user-facing indicator (e.g., an LED, a loudspeaker, or a vibrating actuator), or an outward-facing signal (e.g., an LED, a loudspeaker). The image displays of each optical assembly 180A, 180B are driven by the image display driver 442. In some example configurations, the output components of the eyewear device 100 further include additional indicators such as audible elements (e.g., loudspeakers), tactile components (e.g., an actuator such as a vibratory motor to generate haptic feedback), and other signal gen-

erators. For example, the device 100 may include a user-facing set of indicators, and an outward-facing set of signals. The user-facing set of indicators are configured to be seen or otherwise sensed by the user of the device 100. For example, the device 100 may include an LED display positioned so the user can see it, a one or more speakers positioned to generate a sound the user can hear, or an actuator to provide haptic feedback the user can feel. The outward-facing set of signals are configured to be seen or otherwise sensed by an observer near the device 100. Similarly, the device 100 may include an LED, a loudspeaker, or an actuator that is configured and positioned to be sensed by an observer.

[0085] The input components of the eyewear device 100 may include input components (e.g., a touch screen or touchpad 181 configured to receive alphanumeric input, a photo-optical keyboard, or other alphanumeric-configured elements), pointer-based input components (e.g., a mouse, a touchpad, a trackball, a joystick, a motion sensor, or other pointing instruments), tactile input components (e.g., a button switch, a touch screen or touchpad that senses the location, force or location and force of touches or touch gestures, or other tactile-configured elements), and audio input components (e.g., a microphone), and the like. The mobile device 401 and the server system 498 may include alphanumeric, pointer-based, tactile, audio, and other input components.

[0086] In some examples, the eyewear device 100 includes a collection of motion-sensing components referred to as an inertial measurement unit 472 (which may be duplicated and incorporated into a pair of SoCs). The motion-sensing components may be micro-electro-mechanical systems (MEMS) with microscopic moving parts, often small enough to be part of a microchip. The inertial measurement unit (IMU) 472 in some example configurations includes an accelerometer, a gyroscope, and a magnetometer. The accelerometer senses the linear acceleration of the device 100 (including the acceleration due to gravity) relative to three orthogonal axes (x, y, z). The gyroscope senses the angular velocity of the device 100 about three axes of rotation (pitch, roll, yaw). Together, the accelerometer and gyroscope can provide position, orientation, and motion data about the device relative to six axes (x, y, z, pitch, roll, yaw). The magnetometer, if present, senses the heading of the device 100 relative to magnetic north. The position of the device 100 may be determined by location sensors, such as a GPS unit 473, one or more transceivers to generate relative position coordinates, altitude sensors or barometers, and other orientation sensors (which may be duplicated and incorporated into a pair of SoCs). Such positioning system coordinates can also be received over the wireless connections 425, 437 from the mobile device 401 via the low-power wireless circuitry 424 or the high-speed wireless circuitry 436.

[0087] The inertial management unit (IMU) 472 may include or cooperate with a digital motion processor or programming that gathers the raw data from the components and compute a number of useful values about the position, orientation, and motion of the device 100. For example, the acceleration data gathered from the accelerometer can be integrated to obtain the velocity relative to each axis (x, y, z); and integrated again to obtain the position of the device 100 (in linear coordinates, x, y, and z). The angular velocity data from the gyroscope can be integrated to obtain the position of the device 100 (in spherical coordinates). The

programming for computing these useful values may be stored in memory 434 and executed by the high-speed processor 432 of the eyewear device 100.

[0088] The eyewear device 100 may optionally include additional peripheral sensors, such as biometric sensors, specialty sensors, or display elements integrated with eyewear device 100. For example, peripheral device elements may include any I/O components including output components, motion components, position components, or any other such elements described herein. For example, the biometric sensors may include components to detect expressions (e.g., hand expressions, facial expressions, vocal expressions, body gestures, or eye tracking), to measure bio signals (e.g., blood pressure, heart rate, body temperature, perspiration, or brain waves), or to identify a person (e.g., identification based on voice, retina, facial characteristics, fingerprints, or electrical bio signals such as electroencephalogram data), and the like.

[0089] The mobile device 401 may be a smartphone, tablet, laptop computer, access point, or any other such device capable of connecting with eyewear device 100 using both a low-power wireless connection 425 and a high-speed wireless connection 437. Mobile device 401 is connected to server system 498 and network 495. The network 495 may include any combination of wired and wireless connections.

[0090] The eyewear system 400, as shown in FIG. 4, includes a computing device, such as mobile device 401, coupled to an eyewear device 100 over a network 495. The eyewear system 400 includes a memory for storing instructions and a processor for executing the instructions. Execution of the instructions of the eyewear system 400 by the processor 432 configures the eyewear device 100 to cooperate with the mobile device 401, and also with another eyewear device 100B over the network 495. The eyewear system 400 may utilize the memory 434 of the eyewear device 100 or the memory elements 540A, 540B, 540C of the mobile device 401 (FIG. 5).

[0091] Any of the functionality described herein for the eyewear device 100, the mobile device 401, and the server system 498 can be embodied in one or more computer software applications or sets of programming instructions, as described herein. According to some examples, “function,” “functions,” “application,” “applications,” “instruction,” “instructions,” or “programming” are program(s) that execute functions defined in the programs. Various programming languages can be employed to develop one or more of the applications, structured in a variety of manners, such as object-oriented programming languages (e.g., Objective-C, Java, or C++) or procedural programming languages (e.g., C or assembly language). In a specific example, a third-party application (e.g., an application developed using the ANDROID™ or IOS™ software development kit (SDK) by an entity other than the vendor of the particular platform) may include mobile software running on a mobile OS such as IOS™, ANDROID™, WINDOWS® Phone, or another mobile OSs. In this example, the third-party application can invoke API calls provided by the OS to facilitate functionality described herein.

[0092] Hence, a machine-readable medium may take many forms of tangible storage medium. Non-volatile storage media include, for example, optical or magnetic disks, such as any of the storage devices in any computer devices or the like, such as may be used to implement the client device, media gateway, transcoder, etc. shown in the draw-

ings. Volatile storage media include dynamic memory, such as main memory of such a computer platform. Tangible transmission media include coaxial cables; copper wire and fiber optics, including the wires that comprise a bus within a computer system. Carrier-wave transmission media may take the form of electric or electromagnetic signals, or acoustic or light waves such as those generated during radio frequency (RF) and infrared (IR) data communications. Common forms of computer-readable media therefore include for example: a floppy disk, a flexible disk, hard disk, magnetic tape, any other magnetic medium, a CD-ROM, DVD or DVD-ROM, any other optical medium, punch cards paper tape, any other physical storage medium with patterns of holes, a RAM, a PROM and EPROM, a FLASH-EPROM, any other memory chip or cartridge, a carrier wave transporting data or instructions, cables or links transporting such a carrier wave, or any other medium from which a computer may read programming code or data. Many of these forms of computer readable media may be involved in carrying one or more sequences of one or more instructions to a processor for execution.

[0093] FIG. 5 is a high-level functional block diagram of an example mobile device 401. Mobile device 401 includes a flash memory 540A which stores programming to be executed by the CPU 530. The mobile device 401 also may include a camera 570 that comprises at least two visible-light cameras (first and second visible-light cameras with overlapping fields of view) or at least one visible-light camera and a depth sensor with substantially overlapping fields of view. Flash memory 540A may further include multiple images or video, which are generated via the camera 570.

[0094] As shown, the mobile device 401 includes an image display 580, a mobile display driver 582 to drive the image display 580, and a display controller 584 to control the image display 580. In the example of FIG. 5, the image display 580 includes a user input layer 591 (e.g., a touchscreen) that is layered on top of or otherwise integrated into the screen used by the image display 580. Examples of touchscreen-type mobile devices that may be used include (but are not limited to) a smart phone, a personal digital assistant (PDA), a tablet computer, a laptop computer, or other portable device. However, the structure and operation of the touchscreen-type devices is provided by way of example; the subject technology as described herein is not intended to be limited thereto. For purposes of this discussion, FIG. 5 therefore provides a block diagram illustration of the example mobile device 401 with a user interface that includes a touchscreen input layer 591 for receiving input (by touch, multi-touch, or gesture, and the like, by hand, stylus, or other tool) and an image display 580 for displaying content.

[0095] As shown in FIG. 5, the mobile device 401 includes at least one digital transceiver (XCVR) 510, shown as WWAN XCVRs, for digital wireless communications via a wide-area wireless mobile communication network. The mobile device 401 also includes additional digital or analog transceivers, such as short-range transceivers (XCVRs) 520 for short-range network communication, such as via NFC, VLC, DECT, ZigBee, BLUETOOTH®, or WI-FI®. For example, short range XCVRs 520 may take the form of any available two-way wireless local area network (WLAN) transceiver of a type that is compatible with one or more

standard protocols of communication implemented in wireless local area networks, such as one of the WI-FI® standards under IEEE 802.11.

[0096] To generate location coordinates for positioning of the mobile device 401, the mobile device 401 can include a global positioning system (GPS) receiver. Alternatively, or additionally the mobile device 401 can utilize either or both the short range XCVRs 520 and WWAN XCVRs 510 for generating location coordinates for positioning. For example, cellular network, WI-FI®, or BLUETOOTH® based positioning systems can generate very accurate location coordinates, particularly when used in combination. Such location coordinates can be transmitted to the eyewear device over one or more network connections via XCVRs 510, 520.

[0097] The transceivers 510, 520 (i.e., the network communication interface) conforms to one or more of the various digital wireless communication standards utilized by modern mobile networks. Examples of WWAN transceivers 510 include (but are not limited to) transceivers configured to operate in accordance with Code Division Multiple Access (CDMA) and 3rd Generation Partnership Project (3GPP) network technologies including, for example and without limitation, 3GPP type 2 (or 3GPP2) and LTE, at times referred to as “4G.” The transceivers may also incorporate broadband cellular network technologies referred to as “5G.” For example, the transceivers 510, 520 provide two-way wireless communication of information including digitized audio signals, still image and video signals, web page information for display as well as web-related inputs, and various types of mobile message communications to/from the mobile device 401.

[0098] The mobile device 401 further includes a microprocessor that functions as a central processing unit (CPU) 530. A processor is a circuit having elements structured and arranged to perform one or more processing functions, typically various data processing functions. Although discrete logic components could be used, the examples utilize components forming a programmable CPU. A microprocessor for example includes one or more integrated circuit (IC) chips incorporating the electronic elements to perform the functions of the CPU. The CPU 530, for example, may be based on any known or available microprocessor architecture, such as a Reduced Instruction Set Computing (RISC) using an ARM architecture, as commonly used today in mobile devices and other portable electronic devices. Of course, other arrangements of processor circuitry may be used to form the CPU 530 or processor hardware in smart-phone, laptop computer, and tablet.

[0099] The CPU 530 serves as a programmable host controller for the mobile device 401 by configuring the mobile device 401 to perform various operations, for example, in accordance with instructions or programming executable by CPU 530. For example, such operations may include various general operations of the mobile device, as well as operations related to the programming for applications on the mobile device. Although a processor may be configured by use of hardwired logic, typical processors in mobile devices are general processing circuits configured by execution of programming.

[0100] The mobile device 401 includes a memory or storage system, for storing programming and data. In the example, the memory system may include a flash memory 540A, a random-access memory (RAM) 540B, and other

memory components **540C**, as needed. The RAM **540B** serves as short-term storage for instructions and data being handled by the CPU **530**, e.g., as a working data processing memory. The flash memory **540A** typically provides longer-term storage.

[0101] Hence, in the example of mobile device **401**, the flash memory **540A** is used to store programming or instructions for execution by the CPU **530**. Depending on the type of device, the mobile device **401** stores and runs a mobile OS through which specific applications are executed. Examples of mobile OSs include Google Android, Apple IOS (for iPhone or iPad devices), Windows Mobile, Amazon Fire OS, RIM BlackBerry OS, or the like.

[0102] Finally, the mobile device **401** may further include an inertial measurement unit (IMU) **572** which, in some example configurations, may include an accelerometer, a gyroscope, and a magnetometer.

[0103] The processor **432** within the eyewear device **100** may construct a map of the environment surrounding the eyewear device **100**, determine a location of the eyewear device within the mapped environment, and determine a relative position of the eyewear device to one or more objects in the mapped environment. The processor **432** may construct the map and determine location and position information using a simultaneous localization and mapping (SLAM) algorithm applied to data received from one or more sensors. In the context of augmented reality, a SLAM algorithm is used to construct and update a map of an environment, while simultaneously tracking and updating the location of a device (or a user) within the mapped environment. The mathematical solution can be approximated using various statistical methods, such as particle filters, Kalman filters, extended Kalman filters, and covariance intersection.

[0104] Sensor data includes images received from one or both of the cameras **114A**, **114B**, distance(s) received from a laser range finder, position information received from a GPS unit **473**, or a combination of two or more of such sensor data, or from other sensors providing data useful in determining positional information.

[0105] FIG. 6 is a partial block diagram of an eyewear device **100** incorporating a first SoC **602A** and a second SoC **602B** in accordance with one example. The first SoC **602A** is positioned within a left temple portion **110A** along with a memory **604A** (e.g., flash memory), a battery **606A**, an IMU **472A**, a camera **114A**, and display components **608A**. The second SoC **602B** is positioned within a right temple portion **110B** along with a memory **604B** (e.g., flash memory), a battery **606B**, an IMU **472B**, a camera **114B**, and display components **608B**. The first SoC **602A** is coupled to the second SoC via a communications bus within frame **105** for communications therebetween.

[0106] Although illustrated in the left temple portion **110A**, one or more of the first SoC **602A**, memory **604A**, battery **606A**, and display components **608A** may be positioned in the frame **105** adjacent the left temple portion **110A** (i.e., on the left lateral side **170A**) or in the temple **125A**. Additionally, although illustrated in the right temple portion **110B**, one or more of the second SoC **602B**, memory **604B**, battery **606B**, and display components **608B** may be positioned in the frame **105** adjacent the right temple portion **110B** (i.e., on the right lateral side **170B**) or the temple **125B**. Furthermore, although two memories **604A**, **B**, batteries **606A**, **B**, and display components **608A**, **B** are illus-

trated, fewer or more memories, batteries, and display components may be incorporated. For example, a single battery **606** may power both SoCs **602A**, **B** and SoCs **602A**, **B** may access three or more memories **604** for performing various operations.

[0107] In one example, both SoCs **602** incorporate the same or substantially similar components and component layouts. Thus, their total processing resources are equivalent. In accordance with this example, the first SoC **602A** is at least substantially identical to the second SoC (i.e., they are identical or have an overlap in components or processing resources of 95% or greater). Through the use of dual SoCs **602** (one positioned on one side of the eyewear device **100** and the other on the other side of the eyewear device) cooling is effectively distributed throughout the eyewear device **100** with one side of the eyewear device providing passive cooling for one SoC **602** and the other side of the eyewear device providing passive cooling for the other SoC **602**.

[0108] In one example, the eyewear device **100** has a thermal passive cooling capacity per temple of approximately 3 Watts. The display **608** on each side (e.g., a projection LED display) utilizes approximately 1-2 Watts. Each SoC **602** is designed to operate at less than approximately 1.5 Watts (e.g., 800-1000 mW; unlike the approximately 5 Watts typically used for an SoC in a mobile phone), which enables suitable cooling of the electronics on each side of the eyewear device **100** utilizing passive cooling through the frame **105**, temple portions **110A**, temples **125A**, or a combination thereof. By incorporating two SoCs **602** (positioned on opposite sides of the eyewear device **100** to take advantage of the unique passive cooling capacity presented by the eyewear device **100**), computational power meeting or exceeding that available in a conventional mobile device (which utilizes an SoC operating at 5 Watts of power dissipated) is achievable.

[0109] Incorporating the same or similar components and component layouts in each SoC enables flexibility in distributing processing workload between the two SoCs **602**. In one example, processing workload is distributed based on adjacent components. In accordance with this example, each SoC may drive a respective camera and a display, which may be desirable from an electrical standpoint. In another example, processing workload is distributed based on functionality. In accordance with this example, one SoC **602** may act as a sensor hub (e.g., do all computer vision, CV, and machine learning, ML, processing plus video encoding) and the other SoC **602** may run application logic, audio and video rendering functions, and communications (e.g., WI-FI®, BLUETOOTH®, 4G/5G, etc.). Distributing processing workload based on functionality may be desirable from a privacy perspective. For example, processing sensor information with one SoC and WI-FR with the other enables an implementation where private data such as camera images may be prevented from leaving the eyewear device unnoticed by not allowing such sensor information to be sent from the SoC doing sensor processing to the SoC managing communications. In another example, as described in further detail below, processing workload can be shifted based on SoC temperature, instructions per second, of the like.

[0110] FIGS. 7-9 illustrate flowcharts **700/800/900** for implementing dual SoCs in an eyewear device in sample configurations. Although the steps are described with reference to eyewear device **100**, other suitable eyewear devices

in which one or more steps of the flowcharts **700/800/900** can be practiced will be understood by one of skill in the art from the description herein. Additionally, it is contemplated that one or more of the steps shown in FIGS. **7-9**, and described herein may be omitted, performed simultaneously or in series, performed in an order other than illustrated and described, or performed in conjunction with additional steps.

[0111] FIG. **7** is a flowchart **700** of example steps for performing operations on eyewear with a first SoC and a second SoC. At block **702**, a first SoC (e.g., SoC **602A**) performs a first set of operations. At block **704**, a second SoC (e.g., SoC **602B**) perform a second set of operations. The first and second sets of operations may be distributed therebetween for performance on the respective SoC based on adjacent components, functionality, current workload processing (e.g., as determined based on temperature as described in the example steps of flow chart **700** or instructions per second), or a combination thereof.

[0112] At block **706**, the eyewear device **100** monitors temperatures of the first and second SoCs. In one example, each SoC includes an integrated thermistor for measuring temperature. In accordance with this example, one SoC may be designated as a primary SoC and the other SoC may be designated as a replica SoC. The primary SoC may monitor its own temperature via a respective integrated thermistor and may monitor the temperature of the replica SoC by periodically requesting temperature readings from the replica SoC (which monitor its own temperature via a respective integrated thermistor).

[0113] At block **708**, the eyewear device **100** shifts processing workloads between the first and second sets of operations performed on respective SoC to balance temperature (which effectively distributes processing workload). In examples including a primary SoC and a replica SoC, the primary SoC manages the assignments of workloads to itself and to the replica SoC to maintain a relatively even distribution between the SoCs. In one example, when one of the SoC has a temperature that is above 10% of the temperature of the other SoC, the primary SoC reallocates processing workload from the SoC with the higher temperature to the SoC with the lower temperature until the temperature difference is less than 5%. Processing instructions performed by each of the SoC may be assigned assignability values from 1 to 10 with 1 never being assignable and 10 always being assignable. When shifting processing workloads, the primary SoC may initially shift instructions with assignability values of 10, then 9, 8, etc. The steps for blocks **706** and **708** are continuously repeated to maintain even thermal distribution.

[0114] FIG. **8** is a flowchart **800** of example steps of a method for balancing processing workloads on an eyewear device between a first SoC and a second SoC. At block **802**, a first SoC performs computer vision, machine learning, and video encoding. At block **804**, a second SoC runs application logic, perform rendering functions, and manage wireless communications.

[0115] FIG. **9** is a flowchart **900** of example steps of another method of balancing processing workloads on an eyewear device between a first SoC and a second SoC. At block **902**, the first SoC drives a first camera and a first display. At block **904**, a second SoC drives a second camera and a second display.

[0116] FIG. **10A** depicts a strategy for dividing processing workload between a first SoC **602A** and a second SoC **602B**

of an eyewear device **100**. This strategy balances power from a first side of the eyewear device **100** (e.g., left) to a second side of the eyewear device **100** (e.g., right), reduces interconnect complexity (e.g., with wireless subsystem managed by the second SoC **602B**), and can be dynamically allocated between the left and right based on thermal load, processing requirements, or a combination thereof.

[0117] The first SoC **602A** is connected to the second SoC **602B**, e.g., by an inter-processor communication bus **603** such as PCI Express, SDIO, USB, etc. Communication with each SoC and between the SoCs are in accordance with an inter-process communication (IPC) protocol. A first memory **604A** is incorporated into the first SoC **602A** and a second memory **604B** is incorporate into the second SoC **602B**. In the illustrated example, a first camera **114A**, a second camera **114B**, a first display **608A**, and a second display **608B** of the eyewear device are coupled directly to the first SoC **602A**. The second SoC **602B** is coupled to a low power microprocessor **422** and to wireless circuitry **424/436**.

[0118] In sample configurations, each SoC **602A** and **602B** operates at approximately 1.5 Watt or less (e.g., 800-850 mW). In one example, the first SoC **602A** is responsible for running an operating system (OS) on a CPU (100 mW), capturing images with an ISP/DSP (200 mW), rendering still and video images on a graphics processing unit (GPU) (200 mW), and running various algorithms on the CPU, dedicated hardware accelerated logic blocks (e.g., a disparity mapper), and the digital signal processor (DSP) (350 mW) for a total of 850 mW of power allocated to the first side of the eyewear device. On the other hand, the second SoC **602B** may be responsible for running an OS on a CPU (100 mW), wireless connectivity using the CPU (100 mW), running various algorithms on the CPU and a DSP (300 mW), and running various algorithms on the CPU, a GPU, and the DSP (300 mW) for a total of 800 mW of power allocated to the second side of the eyewear device. This implementation is well below the target of approximately 2-3 W of passive thermal distribution per side of the eyewear device **100**.

[0119] FIG. **10B** depicts a split image capture and rendering strategy for dividing processing workload between a first SoC **602A** and a second SoC **602B** of an eyewear device **100**. The first SoC **602A** is coupled directly to the first and second cameras **114A**, **114B** and the second SoC **602B** is coupled directly to the first and second displays **608A**, **B**, the low power microprocessor **422** and the wireless circuitry **424/436**. The first SoC **602A** may be connected to the second SoC **602B** by an inter-processor communication bus **603** such as PCI Express, SDIO, USB, etc., and power can be allocated dynamically between the left and right based on thermal load, processing requirements, or a combination thereof. This strategy includes a layer of security by coupling the cameras **114A**, **114B** directly to a first SoC **602A** and not directly to the second SoC **602B** or the wireless circuitry **424/436**.

[0120] Each SoC operates at approximately 1.5 Watts or less (e.g., 950-1000 mW). In one example, the first SoC **602A** is responsible for running an OS on a CPU (100 mW), capturing images with an ISP/DSP (200 mW), various algorithms on the CPU, dedicated hardware accelerated logic blocks (e.g., a disparity mapper), and the DSP (350 mW), and running various algorithms on the CPU, a GPU, and the DSP (300 mW) for a total of 950 mW of power allocated to the first side of the eyewear device. The second SoC **602B** is responsible for running an OS on a CPU (100

mW), wireless connectivity using the CPU (100 mW), rendering images on the GPU (200 mW), running various algorithms on the CPU and a DSP (300 mW), and running various algorithms on the CPU, the GPU, and the DSP (300 mW) for a total of 1000 mW of power allocated to the second side of the eyewear device. This implementation is well below the target of approximately 2-3 W of passive thermal distribution per side of the eyewear device **100**.

[0121] FIG. **10C** depicts a left-right component strategy for dividing processing workload between a first SoC **602A** and a second SoC **602B** of an eyewear device **100**. The first SoC **602A** is coupled directly to the first camera **114A** and the first display **608A** and the second SoC **602B** is coupled directly to the second camera **114B** and the second display **608B**, the low power microprocessor **422** and the wireless circuitry **424/436**. The first SoC **602A** may be connected to the second SoC **602B** by an inter-processor communication bus **603** such as PCI Express, SDIO, USB, etc., and power can be allocated dynamically between the left and right based on thermal load, processing requirements, or a combination thereof.

[0122] Each SoC operates at approximately 1.5 Watts or (e.g., 950-1000 mW). In one example, the first SoC **602A** is responsible for running an OS on a CPU (100 mW), capturing images with an ISP/DSP (100 mW), rendering images on the GPU (100 mW), various algorithms on the CPU, dedicated hardware accelerated logic blocks (e.g., a disparity mapper), and the DSP (350 mW), and running various algorithms on the CPU, the GPU, and the DSP (300 mW) for a total of 950 mW of power allocated to the first side of the eyewear device. The second SoC **602B** is responsible for running an OS on a CPU (100 mW), wireless connectivity using the CPU (100 mW), capturing images with an ISP/DSP (100 mW), rendering images on the GPU (100 mW), running various algorithms on the CPU and a DSP (300 mW), and running various algorithms on the CPU, the GPU, and the DSP (300 mW) for a total of 1000 mW of power allocated to the second side of the eyewear device. This implementation is well below the target of approximately 2-3 W of passive thermal distribution per side of the eyewear device **100**.

[0123] FIG. **11** depicts an example of a containerized OS implemented on a computing system **1100B** for use in an electronic device (e.g., an AR eyewear device in the illustrated example). Unlike traditional OSs, the containerized OS coordinates services instead of processes. The computing system **1100B** includes a first processing system **AP1** (first application processor **1152A** such as a first SoC described herein) and a second processing system **AP2** (second application processor **1152B** such as a second SoC described herein). Although this example is described with reference to a processing system having two application processors **1152A**, **1152B**, one of skill in the art will understand how to apply the containerized OS approach to systems with more (e.g., four) or fewer (e.g., one) processing systems. Although the example containerized OS depicted in FIG. **11** and described herein utilizes a hypervisor as a system isolation manager for supporting isolation and communication between virtual machines, one of skill in the art will understand how to implement isolation and communication using a system isolation manager for supporting isolation and communication at a container level using a container manager such as Docker available from Docker, Inc. of Palo Alto, California, USA.

[0124] Each application processor **1152A**, **1152B** may include a respective hypervisor **1154** (e.g., hypervisor **1154A** running on application processor **1152A** and hypervisor **1154B** running on application processor **1152B**). In an example, hypervisor **1154A** and hypervisor **1154B** are at least substantially identical so that a SCVM can run on the hypervisor **1154** on either application processor **1152**. In an example, the hypervisor **1154** is configured to start (spawn) and stop each SCVM, delegate peripheral access to the SCVMs, arbitrate access to shared resources, enforce bandwidth limits for access to the shared resources, or a combination thereof.

[0125] One or more SCVMs **1156** run on the hypervisor **1154** of a respective application processor **1152**. Each SCVM **1156** includes its own OS and is configured to provide at least one service. Additionally, a supervisor OS (not shown) may run on each of the hypervisors **1154**.

[0126] In the illustrated example, on the first application processor **1152A**, a first SCVM **1156A** includes an OS **1158A** and is configured to provide an application service **1160A1** (e.g., a social media application) and a BLUETOOTH® synchronization service **1160A2**. A second SCVM **1156B** includes an OS **1158B** and is configured to provide a transcoding service **1160B**. A third SCVM **1156C** includes a first OS **1158C1** that is configured to provide a lens core service **1160C1** and a second OS **1158C2** that is configured to provide a compositor service **1160C2**. Where a SCVM provides more than one service, a kernel is shared by the containers providing those services for managing processing by the containers on the SCVM.

[0127] On the second application processor **1152B**, a fourth SCVM **1156D** includes an OS **1158D** and is configured to provide a video input/output (VIO) service **1160D**. A fifth SCVM **1156E** includes an OS **1158E** and is configured to provide a scanning service **1160E**. A sixth SCVM **1156F** includes an OS **1158F** and is configured to provide a handtracking service **1160F**. A seventh SCVM **1156G** includes an OS **1158G** and is configured to provide a camera service **1160G**.

[0128] In one example, each SCVM includes at least one resource budget (not shown). The resource budget specifies the maximum amount of a particular resource the SCVM can request/utilize. Example resources include, by way of non-limiting example, allocation of memory **434**, bandwidth of processor **432**, etc. (e.g., reserve 200 MB of RAM to VIO service use, no overcommit at the service level 5, guarantee CPU, memory, network, and I/O bandwidth).

[0129] In examples where each SCVM includes two or more resource budgets, the SCVM may select the appropriate resource budget (e.g., responsive to the current mode of operation of the electronic device or application processors). For example, one of the modes of operation may be for an image acquisition mode (in which the camera service **1160G** and VIO service **1160D** require more resources) and another mode of operation may be for an image rendering mode (in which the lens core service **1160C1** and the compositor service **1160C2** require more resources).

[0130] The services **1160A2** and **1160B-F** provide services that support and enable functions of the application service **1160A1**. The application processor **1152** may be associated with one or more digital signal processors (e.g., application processor **1152A** is associated with aDSP **1162A1** and cDSP **1162B1** and application processor **1152B** is associated with aDSP **1162A2** and cDSP **1162B2**). The

scan service **1160E** coordinates access to hardware resources such as the digital signal processors (e.g., aDSP **1162A2** and cDSP **1162B2**). The application service **1160A1** receives images from cameras **114A**, **114B** (e.g., via camera service **1160G**) and presents images via projectors **180A**, **180B** (e.g., via lens core service **1160C1** and compositor service **1160C2**).

[0131] The computer system **1100B** may additionally include a communication coprocessor **1122** having a real-time OS **1124** such as a coprocessor available from Nordic Semiconductor, Inc. of Cupertino, California, USA.

[0132] The components/services communicate with one another (inter-application processor and intra-application processor) using an IPC protocol supported by the supervisor OS and the OSs **1158**. The IPC protocol abstracts over the physical location of each service. In other words, from the perspective of a service, an IPC call works the same way whether a callee resides on the caller's SoC, a different SoC, or a coprocessor peer like a Nordic unit or a Snapdragon xDSP unit.

[0133] In one example, the IPC protocol is a data interchange format protocol such as Cap'n Proto available from sandstorm.io, protobuf, or other data exchange format. A subset of connections is illustrated in FIG. **11**; however, essentially any service/component can communicate with any other service/component using the IPC protocol. In addition to inter-application processor/SoC and intra-application processor/SoC, services can communicate with other resources (e.g., aDSP **1162A**, cDSP **1162B**, and Nordic **1122**) by sending messages utilizing the IPC protocol.

[0134] FIGS. **12A-12C** include flowcharts **1200/1220/1240** for implementing an OS on a processing system of computing devices such as a single SoC or dual SoCs in a computing device, e.g., an eyewear device of the type described above. Although the steps are described with reference to eyewear device **100**, other suitable devices in which one or more steps of the flowcharts **1200/1220/1240** can be implemented will be understood by one of skill in the art from the description herein. Additionally, it is contemplated that one or more of the steps shown in FIGS. **12A-12C**, and described herein, may be omitted, performed simultaneously or in series, performed in an order other than illustrated and described, or performed in conjunction with additional steps. Furthermore, although the example methods depicted in FIGS. **12A-12C** and described herein utilize a hypervisor as a system isolation manager for supporting isolation and communication between virtual machines, one of skill in the art will understand how to implement isolation and communication using a system isolation manager for supporting isolation and communication at a container level using a container manager such as Docker available from Docker, Inc. of Palo Alto, California, USA.

[0135] FIG. **12A** is a flowchart **1200** of example steps for performing operations on a computing system implemented as a SoC. At block **1202**, a processing system runs a hypervisor. The hypervisor has a supervisor OS. The hypervisor runs on each of at least one processing system where each processing system may be implemented as one or more SoCs (e.g., on each of first SoC **602A** and second SoC **602B**).

[0136] At block **1204**, the hypervisor spawns a first SCVM having a first OS configured to run on the hypervisor. The first SCVM is configured to provide a first service (e.g., an application service). In one example, the first SCVM

includes a first container and the first service runs in the first container. In one example, the hypervisor is configured to spawn particular SCVMs/services on particular SoCs. In other examples, the hypervisor is configured to dynamically spawn the SCVMs on the SoCs, e.g., to distribute processing and balance thermal loads. Where the at least one processor is one or more SoCs, a SoC spawns the first SCVM.

[0137] At block **1206**, the hypervisor spawns a second SCVM having a second OS configured to run on the hypervisor. The second SCVM is configured to provide a second service (e.g., a compositor service). In one example, the second SCVM includes a second container and the second service runs in the second container. Where the at least one processor is one or more SoCs, a SoC spawns the second SCVM.

[0138] At block **1208**, the first SCVM communicates with the second SCVM. The SCVMs communicate with one another via an IPC protocol supported by the supervisor OS of the hypervisor, the first OS, and the second OS. Additionally, the SCVMs may communicate with other components/resources.

[0139] At block **1210**, the computing system manages computing resources of the SCVMs. In an example, the computer system manages computing resources through the IPC protocol. For example, each SCVM may include a resource budget (which is allocated/planned for within the computing system). During operation, an SCVM monitors its resource budget and only requests resources from the computing system (via the IPC protocol) if within the resource budget. The computing system schedules requested services (e.g., using a round-robin with priority levels scheme). In an example, the computing resources meet or exceed the combined resource budgets from all SCVMs. Since the SCVMs only requests resources within their resource budget, there will be adequate resources available to fulfil the requests of all SCVMs without the need to deny or restrict access to services.

[0140] In one example, services that interact with hardware exclusively (e.g., compositor service, camera services) are provided with direct access to that hardware via virtual machine (VM) pass-through, which removes the need for paravirtualization and device emulation overhead.

[0141] FIG. **12B** is a flowchart **1220** of additional example steps for performing operations on a computing system implemented as a SoC. At block **1222**, the computing system operates in a first mode (e.g., an image acquisition mode). In an example, the computing system has multiple operating modes and one or more (e.g., all) SCVMs have corresponding operating modes. The operating mode of each SCVM in accordance with this example has an associated defined resource budget (e.g., maintained in a lookup table accessible to the SCVM).

[0142] At block **1224**, the SCVMs operate in the first mode. When in the first mode of operation, each SCVM utilizes resources within constraints imposed by the respective resource budget of that SCVM operating in that mode. The SCVMs may default to the first mode when spawned or the hypervisor may provide a communication to the SCVM (e.g., via IPC protocol) at the time of spawning identifying the first mode.

[0143] At block **1226**, the computing system notifies the SCVMs that it is transitioning to a second mode (e.g., an image projection mode). When the computing system is changing from a first mode to a second mode, the computing

system notifies the SCVMs of the transition (e.g., via IPC protocol) so they can prepare for the transition.

[0144] At block 1228, the computing system transitions to the second mode.

[0145] At block 1230, the SCVMs operate in the second mode. When in the second mode of operation, each SCVM utilizes resources within constraints imposed by the respective resource budget of that SCVM operating in that mode. The SCVMs may transition to operating in the second mode in response to a notification from the computing system (e.g., via IPC protocol) that it is transitioning to the second mode.

[0146] FIG. 12C is a flowchart 1240 of additional example steps for performing operations on a hypervisor. At block 1242, the hypervisor starts and stops the first SCVM and the second SCVM. In an example, the hypervisor starts and stops virtual machine services on nodes in an SoC on demand according to changes in configuration and device state. At block 1244, the hypervisor delegates peripheral access to the first SCVM and the second SCVM. At block 1246, the hypervisor arbitrates access to shared resources. At block 1248, the hypervisor enforces bandwidth limits for access to the shared resources.

[0147] As mentioned above, a multiple system-on-chip (SoC) architecture may be used to optimize processing while expanding the potential thermal envelope of an electronic eyewear device 100. In many applications of the configurations described above, the respective SoCs will need to work together and to operate on the same (or similar) time scale in order to share information during processing. For example, an algorithm running on one SoC may use data collected by another SoC (e.g., camera frames). In order to process the collected data, the timestamps must be very accurate (less than 100 microseconds). However, the respective SoCs are typically independent from each other and operate according to their own time bases without having access to external systems (e.g., WAN systems) to “tune” their internal clocks. For SoCs such as those used in an electronic eyewear device 100 of the type described above where Internet connectivity may not be available to adjust the internal clocks, an alternative synchronization technique is desired.

[0148] While the respective SoCs used to implement the functionality of the electronic eyewear device 100 described above with respect to FIGS. 1-12 are ideally physically separated to optimize passive thermal distribution, the operations of the respective SoCs may need to be closely integrated. In such multi-SoC systems, timestamped data is often produced on one SoC and consumed by another SoC. Occasionally, such data consists of multiple timestamped streams originating in different parts of the system and eventually fused together to produce a new data stream. In these scenarios, it is important to ensure that the clocks used for timestamping of the respective data streams use the same time base. However, the respective SoCs, even if identical, are not guaranteed to have the same time base. For example, conventional SoCs may have their own oscillators driving the internal clock resulting in a small drift caused by variations in oscillator frequency. The SoCs start their internal clock from the time of boot. Unfortunately, as the crystals and clock generator circuits are incorporated into the SoCs, the SoCs may not share an external clock. It is desired to provide a time synchronization technique to

enable the data generated by respective SoCs in an electronic eyewear device 100 of the type described in detail above to be reconciled.

[0149] The time synchronization technique utilized in sample configurations synchronizes independent SoCs by sharing timestamps recorded upon assertion of general purpose input/output (GPIO) connection between the SoCs. In a sample configuration, a primary SoC asserts a shared GPIO connection between the primary SoC and a secondary SoC and records the primary clock timestamp (gpio_primary_time) for the time that the GPIO connection was asserted. Asserting the GPIO connection triggers an interrupt request (IRQ) at the secondary SoC, which records its local timestamp (gpio_local_time) for the time when the IRQ was received. The primary SoC then sends a message with the primary clock timestamp to the secondary SoC over an inter-SoC bus. The secondary SoC uses the primary clock timestamp and its own local timestamp to compute the time offset between the primary clock and its local clock as $\text{time_offset} = \text{gpio_primary_time} - \text{gpio_local_time}$. The secondary SoC then can compute the primary clock's time locally as $\text{primary_time} = \text{local_time} + \text{time_offset}$ and may operate using the primary clock time in synchrony with the primary SoC. This approach requires little firmware overhead to implement but does require a GPIO connection to be established between the primary SoC and the secondary SoC.

[0150] FIG. 13 illustrates a sample configuration for two SoCs including SoC1 1300 and SoC2 1310 connected by an inter-SoC (ISC) interface (e.g., PCIe bus) 1320 that implements, for example, the PCIe protocol to exchange information between the SoC1 1300 and the SoC2 1310. In sample configurations, a shared general purpose input/output (GPIO) connection 1330 is also provided between SoC1 1300 and SoC2 1310. As illustrated, SoC1 1300 receives respective clock signals from a primary clock generator 1340 including a timing crystal (xtal1) 1350. Similarly, SoC2 1310 receives respective clock signals from a secondary clock generator 1360 including a timing crystal (xtal2) 1370. The respective SoCs 1300, 1310 may be the same or different. Generally speaking, the SoCs 1300 and 1310 may have frequencies that are different but that do not change much over time. However, the SoCs 1300 and 1310 generally will have independent time bases as they are driven by different crystal oscillators with different error margins, so the time bases for the SoCs are generally different.

[0151] In a sample configuration, the clocks output by the primary clock generator 1340 and the secondary clock generator 1360 are synchronized in accordance with the sequence diagram illustrated in FIG. 14. As illustrated, when activity is asserted on the shared GPIO connection 1330 between SoC1 1300 and SoC2 1310 by SoC1 1300, a hardware interrupt request (IRQ) is triggered and provided to SoC2 1310 via the GPIO connection 1330. The GPIO can be set to 0 (low voltage) or 1 (high voltage). In this context, “assert the GPIO” means “set the GPIO from 0 to 1” or from low voltage to high voltage. SoC2 1310 detects that the value on the GPIO (i.e., the voltage level) changed and that triggers the interrupt request to record the local time on SoC2 1310. The timing of the assertion of the shared GPIO connection 1330 is recorded by SoC1 1300 by a primary timestamp (gpio_primary_time), and the timing of receipt of the IRQ is recorded by SoC2 1310 by a local timestamp (gpio_local_time). As SoC1 1300 and SoC2 1310 are

directly connected by the GPIO connection 1330, the time latency for the transmission of the IRQ is nominal (a simple pulse would transmit with latency on the order of a few nanoseconds) compared to the latency of the ISC interface 1320, for example. The timestamping functionality may be implemented in hardware or software/firmware within the software stack of the respective SoCs.

[0152] SoC1 1300 next sends a message including the primary timestamp (gpio_primary_time) to SoC2 1310 over the inter-SoC bus 1320. Upon receipt, SoC2 1310 uses the primary timestamp (gpio_primary_time) and its own local timestamp (gpio_local_time) to compute the time offset between the primary clock from primary clock generator 1340 and the secondary clock from secondary clock generator 1360 as $\text{time_offset} = \text{gpio_primary_time} - \text{gpio_local_time}$. SoC2 1310 then can compute the primary clock from the primary clock generator 1340 locally as $\text{primary_time} = \text{local_time} + \text{time_offset}$. Software, firmware, or both on SoC1 1300 and SoC2 may implement this synchronization functionality with little overhead.

[0153] FIG. 15 is a flowchart illustrating the method 1500 for synchronizing two SoCs 1300 and 1310 in sample configurations. For example, the SoCs 1300 and 1310 may be disposed in respective temples of an electronic eyewear device 100 of the type described above where the respective SoCs 1300 and 1310 are connected by an inter-SoC interface 1320 such as a PCIe bus as well as GPIO connection 1330. It will be appreciated that the method 1500 may be partially implemented in software on one or both of the respective SoCs 1300 and 1310 and that either SoC 1300 or 1310 may be the primary or the secondary SoC in this configuration.

[0154] At 1510, SoC1 1300 asserts the shared GPIO connection 1330 to SoC2 1310.

[0155] At 1520, SoC1 1300 records a primary timestamp for the time of assertion of the shared GPIO connection 1330.

[0156] At 1530, the assertion of the shared GPIO connection 1330 triggers an interrupt request at SoC2 1310.

[0157] At 1540, SoC2 1310 records a local timestamp of receipt of the IRQ on the shared GPIO connection 1330.

[0158] At 1550, SoC1 1300 sends a message including the primary timestamp to SoC2 1310 over the ISC interface 1320.

[0159] At 1560, SoC2 1310 computes the clock offset between SoC1 1300 and SoC2 1310 as the difference between the primary timestamp and the local timestamp.

[0160] At 1570, SoC2 1310 computes the primary clock of SoC1 1300 from the primary clock generator 1340 as the sum of the local timestamp and the calculated clock offset. SoC2 1310 may then use the primary clock in synchrony with SoC1 1300.

[0161] This process may be repeated periodically to adjust the timing between the SoCs 1300 and 1310 to adjust for any drift in the clock signals over time. As indicated, the timing of SoC2 1310 may be adjusted in line with the calculated primary clock. The process may be repeated as often as needed to maintain synchronization of the respective SoCs 1300 and 1310.

[0162] In the noted configuration, SoC1 1300 and SoC2 1310 may be synchronized any time a message is communicated via the GPIO connection 1330. The SoC1 1300 may assert the GPIO connection 1330 periodically to keep SoC1 1300 and SoC2 in synchrony as needed.

[0163] It will be further appreciated by those skilled in the art that the GPIO connection 1330 is independent of the communication channel over the ISC interface 1320. Also, it will be appreciated that the GPIO connection 1330 does not require the overhead of Peripheral Component Interconnect Express (PCIe) messaging.

[0164] The clock synchronization techniques described herein may work for different clock rates for different SoCs while allowing the clocks to be synchronized less frequently as the relative clock rates typically remain relatively constant over time. The clock offsets may be captured every few seconds or even less frequently to account for relative clock drift over time. In sample configurations, the clock offsets captured at regular intervals may be estimated by extrapolating the clock offsets to determine when the relative clock rates are expected to differ outside of the accepted clock drift tolerance so that the clocks may be synchronized at time intervals calculated to keep the relative clock rates within the clock drift tolerance with minimal overhead. As an example, if the clock drift tolerance threshold is 100 usec and the relative clock rates drift by 20 usecs per second, then the clocks would need to be synchronized by asserting the GPIO connection 1330 at least once every 5 seconds to remain within the clock drift tolerance.

[0165] As used herein, the term “memory” refers to a machine-readable medium adapted to store data temporarily or permanently and may be taken to include, but not be limited to, random-access memory (RAM), read-only memory (ROM), buffer memory, flash memory, and cache memory. In an example system configuration, to be a single medium, the term “machine-readable medium” should be taken to include a single medium or multiple media (e.g., a centralized or distributed database, or associated caches and servers) able to store instructions for execution by one or more processors. The term “machine-readable medium” also shall be taken to include any medium, or combination of multiple media, that is capable of storing instructions for execution by a machine (e.g., one or more processors), such that the instructions, when executed by one or more processors of the machine, cause the machine to perform any one or more of the methodologies described herein. For example, instructions for executing the method 1500 of FIG. 15 may be stored in a machine-readable medium for execution by one or more of the respective SoCs 1300 and 1310. Accordingly, a “machine-readable medium” refers to a single storage apparatus or device, as well as “cloud-based” storage systems or storage networks that include multiple storage apparatus or devices. The term “machine-readable medium” shall accordingly be taken to include, but not be limited to, one or more data repositories in the form of a solid-state memory (e.g., flash memory), an optical medium, a magnetic medium, other non-volatile memory (e.g., erasable programmable read-only memory (EPROM)), or any suitable combination thereof. The term “machine-readable medium” specifically excludes non-statutory signals per se.

[0166] Furthermore, any machine-readable medium is non-transitory (in other words, not having any transitory signals) in that it does not embody a propagating signal. However, labelling the machine-readable medium “non-transitory” should not be construed to mean that the medium is incapable of movement; the medium should be considered as being transportable from one physical location to another.

Additionally, since the machine-readable medium is tangible, the medium may be considered to be a machine-readable device.

[0167] It will be understood that the terms and expressions used herein have the ordinary meaning as is accorded to such terms and expressions with respect to their corresponding respective areas of inquiry and study except where specific meanings have otherwise been set forth herein. Relational terms such as first and second and the like may be used solely to distinguish one entity or action from another without necessarily requiring or implying any actual such relationship or order between such entities or actions. The terms “comprises,” “comprising,” “includes,” “including,” or any other variation thereof, are intended to cover a non-exclusive inclusion, such that a process, method, article, or apparatus that comprises or includes a list of elements or steps does not include only those elements or steps but may include other elements or steps not expressly listed or inherent to such process, method, article, or apparatus. An element preceded by “a” or “an” does not, without further constraints, preclude the existence of additional identical elements in the process, method, article, or apparatus that comprises the element.

[0168] As used herein, the term “or” may be construed in either an inclusive or exclusive sense. Moreover, plural instances may be provided for resources, operations, or structures described herein as a single instance. Additionally, boundaries between various resources, operations, modules, engines, and data stores are somewhat arbitrary, and particular operations are illustrated in a context of specific illustrative configurations. Other allocations of functionality are envisioned and may fall within a scope of various system configurations of the present disclosure. In general, structures and functionality presented as separate resources in the example configurations may be implemented as a combined structure or resource. Similarly, structures and functionality presented as a single resource may be implemented as separate resources. These and other variations, modifications, additions, and improvements fall within a scope of system configurations of the present disclosure as represented by the appended claims. The specification and drawings are, accordingly, to be regarded in an illustrative rather than a restrictive sense.

[0169] Unless otherwise stated, any and all measurements, values, ratings, positions, magnitudes, sizes, and other specifications that are set forth in this specification, including in the claims that follow, are approximate, not exact. Such amounts are intended to have a reasonable range that is consistent with the functions to which they relate and with what is customary in the art to which they pertain. For example, unless expressly stated otherwise, a parameter value or the like may vary by as much as plus or minus ten percent from the stated amount or range.

[0170] In addition, in the foregoing Detailed Description, it can be seen that various features are grouped together in various examples for the purpose of streamlining the disclosure. This method of disclosure is not to be interpreted as reflecting an intention that the claimed examples require more features than are expressly recited in each claim. Rather, as the following claims reflect, the subject matter to be protected lies in less than all features of any single disclosed example. Thus, the following claims are hereby incorporated into the Detailed Description, with each claim standing on its own as a separately claimed subject matter.

[0171] While the foregoing has described what are considered to be the best mode and other examples, it is understood that various modifications may be made therein and that the subject matter disclosed herein may be implemented in various forms and examples, and that they may be applied in numerous applications, only some of which have been described herein. It is intended by the following claims to claim any and all modifications and variations that fall within the true scope of the present concepts.

[0172] The system configurations illustrated herein are described in sufficient detail to enable those skilled in the art to practice the teachings disclosed. Other system configurations may be used and derived therefrom, such that structural and logical substitutions and changes may be made without departing from the scope of this disclosure. The Detailed Description, therefore, is not to be taken in a limiting sense, and the scope of various system configurations is defined only by the appended claims, along with the full range of equivalents to which such claims are entitled.

[0173] Throughout this specification, plural instances may implement components, operations, or structures described as a single instance. Although individual operations of one or more methods are illustrated and described as separate operations, one or more of the individual operations may be performed concurrently, and nothing requires that the operations be performed in the order illustrated. Structures and functionality presented as separate components in example configurations may be implemented as a combined structure or component. Similarly, structures and functionality presented as a single component may be implemented as separate components. These and other variations, modifications, additions, and improvements fall within the scope of the subject matter herein.

What is claimed is:

1. A method of synchronizing multiple systems-on-chip (SoCs) having independent time bases and connected by an inter-SoC interface that exchanges information among the multiple SoCs and further comprising a shared general purpose input/output (GPIO) connection among the multiple SoCs, the method comprising:

- a first SoC changing a value on the shared GPIO connection to trigger an interrupt request (IRQ);
- the first SoC recording a first timestamp when the value on the shared GPIO connection is changed;
- a second SoC detecting that the value on the shared GPIO connection has changed to trigger an IRQ at the second SoC;
- the second SoC recording a second timestamp of receipt of the IRQ on the shared GPIO connection;
- the first SoC sending a message including the first timestamp over the inter-SoC interface to the second SoC;
- the second SoC calculating a clock offset between the first SoC and the second SoC as a difference between the first timestamp and the second timestamp; and
- the second SoC calculating a clock of the first SoC as a sum of a timestamp of the second SoC and the calculated clock offset.

2. The method of claim 1, further comprising adjusting a clock of the second SoC to the calculated clock of the first SoC.

3. The method of claim 2, further comprising repeating steps (a)-(g) periodically and adjusting the clock of the

second SoC to adjust for any drift in the clock of the first SoC or the clock of the second SoC over time.

4. The method of claim 3, further comprising extrapolating calculated clock offsets to determine when the clock of the second SoC is expected to differ outside of a clock drift tolerance and adjusting the clock of the second SoC to the calculated clock of the first SoC to keep the calculated clock offsets within the clock drift tolerance.

5. The method of claim 1, further comprising repeating steps (a)-(g) each time the IRQ from the first SoC to the second SoC is triggered.

6. The method of claim 1, wherein changing the value on the shared GPIO connection comprises setting the shared GPIO connection from a first logic level to a second logic level.

7. The method of claim 1, wherein at least one of the first SoC recording a first timestamp or the second SoC recording a second timestamp comprises using a hardware timestamping feature of the first SoC or the second SoC.

8. A device comprising:

multiple system-on-chips (SoCs) including a first system-on-chip (SoC) comprising a first clock generator that generates a first clock signal for the first SoC and a second SoC comprising a second clock generator that generates a second clock signal for the second SoC, the first clock generator and the second clock generator having independent time bases;

an inter-SoC interface that exchanges information among the multiple SoCs;

a shared general purpose input/output (GPIO) connection among the multiple SoCs;

a first computer readable medium comprising instructions stored thereon that are executable by the first SoC to cause the first SoC to perform operations for synchronizing the first SoC and the second SoC, the operations including:

- a. the first SoC changing a value on the shared GPIO connection to trigger an interrupt request (IRQ);
- b. the first SoC recording a first timestamp when the value on the shared GPIO connection is changed; and
- c. the first SoC sending a message including the first timestamp to the second SoC over the inter-SoC interface; and

a second computer readable medium comprising instructions stored thereon that are executable by the second SoC to cause the second SoC to perform operations for synchronizing the first SoC and the second SoC, the operations including:

- d. the second SoC detecting that the value on the shared GPIO connection has changed to trigger an IRQ at the second SoC;
- e. the second SoC recording a second timestamp of receipt of the IRQ on the shared GPIO connection;
- f. the second SoC calculating a clock offset between the first SoC and the second SoC as a difference between the first timestamp and the second timestamp; and
- g. the second SoC calculating a clock of the first SoC as a sum of a timestamp of the second SoC and the calculated clock offset.

9. The device of claim 8, wherein the second computer readable medium further comprises instructions stored thereon that are executable by the second SoC to cause the

second SoC to perform operations including adjusting a clock output by the second clock generator to the calculated clock of the first SoC.

10. The device of claim 9, wherein at least one of the first computer readable medium or the second computer readable medium further comprises instructions stored thereon that are executable by at least one of the first SoC or the second SoC to cause the at least one of the first SoC or the second SoC to perform operations including repeating steps (a)-(g) periodically and adjusting the clock signal generated by the second clock generator to adjust for any drift in the first clock signal generated by the first clock generator or in the second clock signal generated by the second clock generator over time.

11. The device of claim 10, wherein the second computer readable medium further comprises instructions stored thereon that are executable by the second SoC to cause the second SoC to perform operations including extrapolating calculated clock offsets to determine when the second clock signal of the second SoC is expected to differ outside of a clock drift tolerance and adjusting the second clock signal of the second SoC to the calculated clock of the first SoC to keep the calculated clock offsets within the clock drift tolerance.

12. The device of claim 8, wherein at least one of the first computer readable medium or the second computer readable medium further comprises instructions stored thereon that are executable by at least one of the first SoC or the second SoC to cause the at least one of the first SoC or the second SoC to perform operations including repeating steps (a)-(g) each time the IRQ from the first SoC to the second SoC is triggered.

13. The device of claim 8, wherein the shared GPIO connection is changed from a first logic level to a second logic level to trigger the IRQ.

14. The device of claim 13, wherein the shared GPIO connection is independent of a communication channel over the inter-SoC interface.

15. The device of claim 8, wherein at least one of the first SoC or the second SoC includes a hardware timestamping feature for recording at least one of the first timestamp or the second timestamp.

16. At least one non-transitory computer readable medium comprising instructions stored thereon that are executable by at least one of a first system-on-chip (SoC) or a second SoC of multiple SoCs having independent time bases to cause the at least one of the first SoC or the second SoC to perform operations for synchronizing the multiple SoCs where the multiple SoCs are connected by an inter-SoC interface that exchanges information among the multiple SoCs and by a shared general purpose input/output (GPIO) connection among the multiple SoCs, the operations including:

- a. a first SoC changing a value on the shared GPIO connection to trigger an interrupt request (IRQ);
- b. the first SoC recording a first timestamp when the value on the shared GPIO connection is changed;
- c. a second SoC detecting that the value on the shared GPIO connection has changed to trigger an IRQ at the second SoC;
- d. the second SoC recording a second timestamp of receipt of the IRQ on the shared GPIO connection;
- e. the first SoC sending a message including the first timestamp over the inter-SoC interface to the second SoC;

- f. the second SoC calculating a clock offset between the first SoC and the second SoC as a difference between the first timestamp and the second timestamp; and
- g. the second SoC calculating a clock of the first SoC as a sum of a timestamp of the second SoC and the calculated clock offset.

17. The medium of claim **16**, further comprising instructions for performing operations including adjusting a clock of the second SoC to the calculated clock of the first SoC.

18. The medium of claim **17**, further comprising instructions for performing operations including repeating steps (a)-(g) periodically and adjusting the clock of the second SoC to adjust for any drift in the clock of the first SoC or the clock of the second SoC over time.

19. The medium of claim **18**, further comprising instructions for performing operations including extrapolating calculated clock offsets to determine when the clock of the second SoC is expected to differ outside of a clock drift tolerance and adjusting the clock of the second SoC to the calculated clock of the first SoC to keep the calculated clock offsets within the clock drift tolerance.

20. The medium of claim **16**, further comprising instructions for performing operations including repeating steps (a)-(g) each time the IRQ from the first SoC to the second SoC is triggered.

* * * * *